

时空数据挖掘

第三讲-时空数据采集、存储及处理

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 11 月 11 日



福州大学
FUZHOU UNIVERSITY

- ① 时空数据的采集
- ② 时空数据的存储
- ③ 时空数据的处理

① 时空数据的采集

以物为中心的感知
以人为中心的感知

② 时空数据的存储

③ 时空数据的处理

（经度、纬度、地址）和时间戳的动态数据的过程

状态、拓扑关系
变化频率
密度等观测值

灾害预警、交通规划等领域的决策基础

一次位置（经度、纬度、时间戳），形成动态

- **定义：**获取兼具空间位置（如经纬度、地址）和时间戳的动态数据的过程
- **核心要素：**
 - 空间维度：地理位置、几何形状、拓扑关系
 - 时间维度：时间点、时间段、变化频率
 - 属性维度：温度、湿度、人口密度等观测值
- **意义：**智慧城市、环境监测、灾害预警、交通规划等领域的决策基础
- **示例：**车辆轨迹数据每秒钟记录一次位置（经度、纬度、时间戳），形成动态路径

- 原始时空数据的主要类型：1

¹WANG S, CAO J, YU P S. Deep Learning for Spatio-Temporal Data Mining: A Survey[J]. IEEE Transactions on Knowledge and Data Engineering, 2022, 34(8): 3681-3700.

是单个事件或对象在特定时空位置的信息（如违法行为）
交网络（**Location-Based Social Network, LBS**）数
示，其中 e 是事件相关的信息， l 表示事件发生位置

- 原始时空数据的主要类型：¹
 - 事件数据 (Event Data)：记录单个事件或对象在特定时空位置的信息（如违法行为、交通事故、基于位置的社交网络 (Location-Based Social Network, LBS) 数据)。通常用三元组 (e, l, t) 表示，其中 e 是事件相关的信息， l 表示事件发生位置， t 事件发生的时间。

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

时空数据的类型

- 原始时空数据的主要类型：¹
 - 事件数据 (Event Data)：记录单个事件或对象在特定时空位置的信息（如违法行为、交通事故、基于位置的社交网络 (Location-Based Social Network, LBS) 数据）。通常用三元组 (e, l, t) 表示，其中 e 是事件相关的信息， l 表示事件发生位置， t 事件发生的时间。
 - 时空轨迹数据 (Trajectory Data)：记录某个对象在连续时间内的运动路径（如车辆移动、配送员轨迹、飞机飞行、动物迁徙）。一个轨迹通常可以用一个元组序列来表示 $\{(l_1, t_1, a_1), (l_2, t_2, a_2), \dots, (l_n, t_n, a_n)\}$ ，其中 a_i 表示第 i 个时空位置纪录的附加属性。

¹WANG S, CAO J, YU P S. Deep Learning for Spatio-Temporal Data Mining: A Survey[J]. IEEE Transactions on Knowledge and Data Engineering, 2022, 34(8): 3681-3700.

时空数据的类型

- 原始时空数据的主要类型：¹
 - 视频：
 - 电子眼：安装在固定位置的监控摄像头采集的数据，带有位置与时间的信息。
 - 个人摄像设备：GoPro、DJI 等运动相机、无人机在录制时会记录 GPS 数据、加速度传感器的数值等。
 - 行车记录仪
 - ...

¹WANG S, CAO J, YU P S. Deep Learning for Spatio-Temporal Data Mining: A Survey[J]. IEEE Transactions on Knowledge and Data Engineering, 2022, 34(8): 3681-3700.

① 时空数据的采集

以物为中心的感知

固定感知

移动感知

以人为中心的感知

② 时空数据的存储

③ 时空数据的处理

- 固定传感器网络:



- 固定传感器网络:
 - 环境监测站

固定感知：技术手段与应用

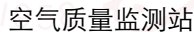
- 固定传感器网络：
 - 环境监测站
 - 气象站：温湿度、气压、风向、风速等



6 要素气象站

- 固定传感器网络：
 - 环境监测站
 - 气象站：温湿度、气压、风向、风速等
 - 空气质量监测站：PM2.5、PM10、CO2 等

空气质量监测站



- 固定传感器网络：
 - 环境监测站
 - 气象站：温湿度、气压、风向、风速等
 - 空气质量监测站：PM2.5、PM10、CO2 等
 - 水文站：水位、流量、污染物浓度等

水文站



水文站

- 固定传感器网络：
 - 环境监测站
 - 数据输出：连续时序数据，如 PM2.5 小时均值

	A	B	C	D	E	F	G
1	time	PM25	PM10	NO2	CO	O3	SO2
17	2015/1/1 15:00	35.1	67.2	33.4	0.9	64.8	18.9
18	2015/1/1 16:00	36.5	73.1	39.9	1	60.1	19.7
19	2015/1/1 17:00	38.4	83.2	58.7	1.3	46.5	23.3
20	2015/1/1 18:00	67	128.5	89.6	1.8	25.7	35.1
21	2015/1/1 19:00	76.5	146.8	106.4	2	11.1	43.2
22	2015/1/1 20:00	93.1	173.2	111.7	2.3	8.9	50.5
23	2015/1/1 21:00	97.4	167.7	111.5	2.3	7.9	50
24	2015/1/1 22:00	93.2	151.6	109.5	2	6.3	49.8
25	2015/1/1 23:00	89	150.8	108.5	2.2	5.5	51.7
26	2015/1/2 00:00	86.1	139.2	104.9	2.1	5.1	46.8
27	2015/1/2 01:00	88	141	108	2.6	5	45
28	2015/1/2 02:00	94	158	119	3.1	8	49
29	2015/1/2 03:00	82	134	108	2.5	6	44
30	2015/1/2 04:00	47	100	92	1.5	4	28
31	2015/1/2 05:00	34	78	86	1.5	4	24
32	2015/1/2 06:00	18	38	62	1	16	18
33	2015/1/2 07:00	12	21	60	0.9	18	13
34	2015/1/2 08:00	9	22	73	0.9	10	12
35	2015/1/2 09:00	10	22	68	0.8	17	11
36	2015/1/2 10:00	7	16	56	0.8	27	14
37	2015/1/2 11:00	11	18	42	0.7	41	18
38	2015/1/2 12:00	16	32	34	0.6	47	22
39	2015/1/2 13:00	10	26	25	0.6	50	22

固定传感器网络数据示例（空气质量时序数据）

固定感知：技术手段与应用

- 固定传感器网络：
 - 环境监测站
 - 数据输出：连续时序数据，如 PM2.5 小时均值
 - 部署特点：位置固定、校准严格、长期观测

	A	B	C	D	E	F	G
1	time	PM25	PM10	NO2	CO	O3	SO2
17	2015/1/1 15:00	35.1	67.2	33.4	0.9	64.8	18.9
18	2015/1/1 16:00	36.5	73.1	39.9	1	60.1	19.7
19	2015/1/1 17:00	38.4	83.2	58.7	1.3	46.5	23.3
20	2015/1/1 18:00	67	128.5	89.6	1.8	25.7	35.1
21	2015/1/1 19:00	76.5	146.8	106.4	2	11.1	43.2
22	2015/1/1 20:00	93.1	173.2	111.7	2.3	8.9	50.5
23	2015/1/1 21:00	97.4	167.7	111.5	2.3	7.9	50
24	2015/1/1 22:00	93.2	151.6	109.5	2	6.3	49.8
25	2015/1/1 23:00	89	150.8	108.5	2.2	5.5	51.7
26	2015/1/2 00:00	86.1	139.2	104.9	2.1	5.1	46.8
27	2015/1/2 01:00	88	141	108	2.6	5	45
28	2015/1/2 02:00	94	158	119	3.1	8	49
29	2015/1/2 03:00	82	134	108	2.5	6	44
30	2015/1/2 04:00	47	100	92	1.5	4	28
31	2015/1/2 05:00	34	78	86	1.5	4	24
32	2015/1/2 06:00	18	38	62	1	16	18
33	2015/1/2 07:00	12	21	60	0.9	18	13
34	2015/1/2 08:00	9	22	73	0.9	10	12
35	2015/1/2 09:00	10	22	68	0.8	17	11
36	2015/1/2 10:00	7	16	56	0.8	27	14
37	2015/1/2 11:00	11	18	42	0.7	41	18
38	2015/1/2 12:00	16	32	34	0.6	47	22
39	2015/1/2 13:00	10	26	25	0.6	50	22

固定传感器网络数据示例（空气质量时序数据）

- 固定传感器网络：
- 遥感卫星：

- 授课教师：柯嵩宇
- 福州大学计算机与大数据学院
- 时空数据挖掘
- 7 / 105

- 固定传感器网络：
- 遥感卫星：
 - 示例：Landsat 系列（30 米分辨率）、Sentinel 系列（10 米分辨率）

Landsat 卫星影像

- 固定传感器网络：
- 遥感卫星：
 - 示例：Landsat 系列（30 米分辨率）、Sentinel 系列（10 米分辨率）



Landsat 卫星影像

- 固定传感器网络：
- 遥感卫星：
 - 示例：Landsat 系列（30 米分辨率）、Sentinel 系列（10 米分辨率）
 - 数据产品：多光谱影像、地表温度、植被指数

多光谱遥感影像

-

多光谱遥感影像

- 溪成數相王裁五公二例 (NAGA E 1110)

- 固定传感器网络:
- 遥感卫星:
- 固定摄像头:

交通监控摄像头



交通监控摄像头

- 固定传感器网络：
- 遥感卫星：
- 固定摄像头：
 - 应用： 交通流量监控、安全防护

交通监控摄像头



交通监控摄像头

- 固定传感器网络：
- 遥感卫星：
- 固定摄像头：
 - 应用：交通流量监控、安全防护
 - 技术挑战：数据存储量大，需边缘计算预处理

A composite of four photographs showing various traffic monitoring camera setups. The top-left image shows a side-view camera mounted on a pole overlooking a multi-lane road with cars. The top-right image shows a large, white, dome-shaped camera mounted on a pole, looking down at a red bus. The bottom-left image shows a camera mounted on a pole, looking down at a road. The bottom-right image shows a complex system with multiple cameras mounted on a single pole, looking in different directions.

交通监控摄像头



交通监控摄像头

- 固定传感器网络：
- 遥感卫星：
- 固定摄像头：
 - 应用：交通流量监控、安全防护
 - 技术挑战：数据存储量大，需边缘计算预处理

- 数据来源：ECMWF 的 ERA5 再分析数据

- 数据来源: ECMWF 的 ERA5 再分析数据

再分析数据

率（全球每小时数据）

分析数据
(全球每小时数据)

- **数据来源:** ECMWF 的 ERA5 再分析数据
- **采集变量:** 风速、温度、气压等 (全球每小时数据)
- **技术方法:**

分析数据
(全球每小时数据)

- 通过 CDS API 批量下载²

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

分析数据
(全球每小时数据)

- 通过 CDS API 批量下载²
- 支持时间范围：1979 年至今

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

分析数据
(全球每小时数据)

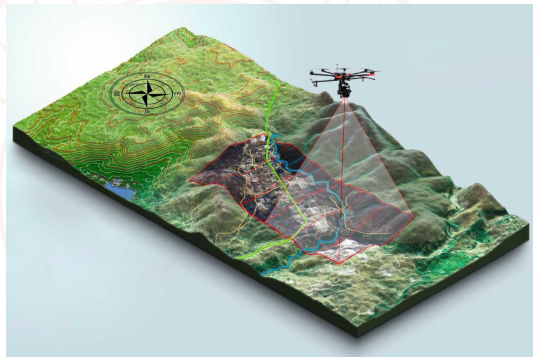
- 通过 CDS API 批量下载²
- 支持时间范围：1979 年至今
- 空间精度：0.25° × 0.25° 网格

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- 再分析数据
等（全球每小时数据）
- 备
模型训练
- /how-to-api

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- 无人机航测：

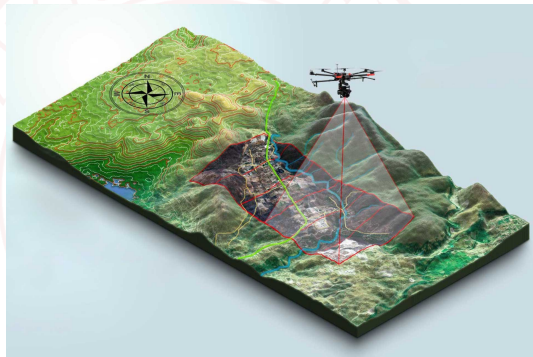


无人机航测示意图

- 无人机航测：
 - 应用：地形测绘、灾害评估、农业监测

无人机航测示意图

- 无人机航测：
 - 应用：地形测绘、灾害评估、农业监测



无人机航测示意图

- 无人机航测：
 - 应用：地形测绘、灾害评估、农业监测
 - 优势：厘米级精度、灵活覆盖

无人机航测示意图



无人机航测示意图

- 无人机航测：
 - 应用：地形测绘、灾害评估、农业监测
 - 优势：厘米级精度、灵活覆盖
 - 数据输出：倾斜摄影模型、多光谱影像

无人机航测示意图

- 普



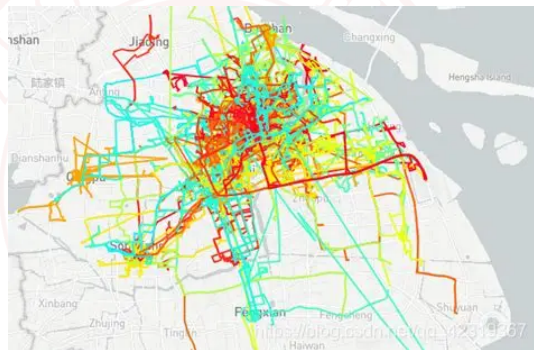
无人机航测示意图

- 无人机航测：
- 车载移动传感器：

- 授课教师：柯嵩宇

移动感知：动态数据采集技术

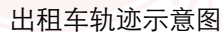
- 无人机航测：
- 车载移动传感器：
 - 示例：出租车 GPS 轨迹（每 10 秒记录位置）



出租车轨迹示意图

- 无人机航测：
- 车载移动传感器：
 - 示例：出租车 GPS 轨迹（每 10 秒记录位置）
 - 数据量：单日千万级轨迹点

出租车轨迹示意图



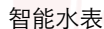
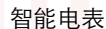
- 无人机航测：
- 车载移动传感器：
 - 示例：出租车 GPS 轨迹（每 10 秒记录位置）
 - 数据量：单日千万级轨迹点
 - 挑战：数据漂移、噪声过滤

轨迹漂移修正示意图

-

轨迹漂移修正示意图

- 无人机航测:
- 车载移动传感器:
- 物联网设备:
 - 智能水表、电表 (带位置标签的时序数据)



- 无人机航测：
- 车载移动传感器：
- 物联网设备：
 - 智能水表、电表（带位置标签的时序数据）

移动感知：动态数据采集技术

- 无人机航测：
- 车载移动传感器：
- 物联网设备：
 - 智能水表、电表（带位置标签的时序数据）^a
 - 新能源汽车 EDR（速度、加速度、油门开度、制动踏板力度等）

^a其实应该算在固定感知中，但是数据传输依托移动网络



智能电表



智能水表



新能源汽车 EDR

- 无人机航测：
- 车载移动传感器：
- 物联网设备：
 - 智能水表、电表（带位置标签的时序数据）^a
 - 新能源汽车 EDR（速度、加速度、油门开度、制动踏板力度等）
 - 物流追踪传感器（温度、湿度、位置）

^a其实应该算在固定感知中，但是数据传输依托移动网络



智能电表



智能水表



新能源汽车 EDR



车载冷链传感器

- 数据采集流程：

- ### ● 数据采集流程:

(时间、经纬度)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

(时间、经纬度)
信号丢失

- 数据采集流程：
 - ① 车载 GPS 设备记录原始轨迹（时间、经纬度）
 - ② 数据清洗：去除漂移点、补偿信号丢失

(时间、经纬度)
信号丢失

- **数据采集流程：**
 - ① 车载 GPS 设备记录原始轨迹（时间、经纬度）
 - ② 数据清洗：去除漂移点、补偿信号丢失
 - ③ 地图匹配：将轨迹点关联至路网

(时间、经纬度)
信号丢失
网

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

(时间、经纬度)
信号丢失
网

(Uber 开源)

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

(时间、经纬度)
信号丢失
网

(Uber 开源)
过程

- 数据采集流程：
 - ① 车载 GPS 设备记录原始轨迹（时间、经纬度）
 - ② 数据清洗：去除漂移点、补偿信号丢失
 - ③ 地图匹配：将轨迹点关联至路网
- 可视化工具：
 - KeplerGL：交互式轨迹热力图（Uber 开源）
 - Mobmap：动态播放轨迹移动过程

(时间、经纬度)
信号丢失
网

(Uber 开源)
过程

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

③ 时空数据的处理

采集工具与技术前沿

- 手机信令数据:

- 授课教师：柯嵩宇

- 手机信令数据：
 - 来源：基站切换记录（时间、位置区码）



被动感知：隐式数据采集

- **手机信令数据：**
 - 来源：基站切换记录（时间、位置区码）
 - 应用：人流密度分析、通勤模式识别



手机信令数据的各种应用

被动感知：隐式数据采集

- 手机信令数据：

- 来源：基站切换记录（时间、位置区码）
- 应用：人流密度分析、通勤模式识别
- 隐私保护：需匿名化、聚合处理



手机信令数据的各种应用

被动感知：隐式数据采集

- **手机信令数据:**
 - 来源: 基站切换记录 (时间、位置区码)
 - 应用: 人流密度分析、通勤模式识别
 - 隐私保护: 需匿名化、聚合处理
- **社交媒体签到:**



- 手机信令数据：
 - 来源：基站切换记录（时间、位置区码）
 - 应用：人流密度分析、通勤模式识别
 - 隐私保护：需匿名化、聚合处理
- 社交媒体签到：
 - 示例：微博、Twitter 带地理位置标签

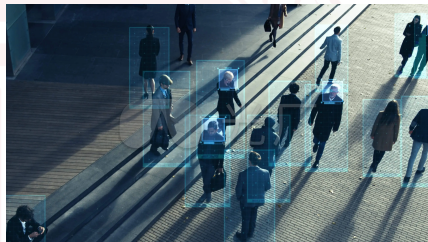
-

- 手机信令数据：
 - 来源：基站切换记录（时间、位置区码）
 - 应用：人流密度分析、通勤模式识别
 - 隐私保护：需匿名化、聚合处理
- 社交媒体签到：
 - 示例：微博、Twitter 带地理位置标签
 - 数据价值：热点区域发现、舆情监控



被动感知：隐式数据采集

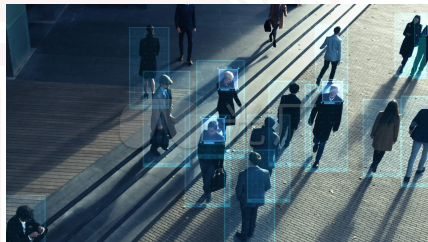
- 手机信令数据：
 - 来源：基站切换记录（时间、位置区码）
 - 应用：人流密度分析、通勤模式识别
 - 隐私保护：需匿名化、聚合处理
- 社交媒体签到：
 - 示例：微博、Twitter 带地理位置标签
 - 数据价值：热点区域发现、舆情监控
- 公共监控视频：



公共监控视频中的人脸检测

被动感知：隐式数据采集

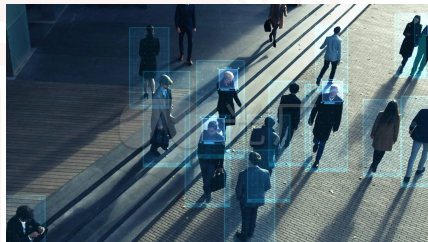
- 手机信令数据：
 - 来源：基站切换记录（时间、位置区码）
 - 应用：人流密度分析、通勤模式识别
 - 隐私保护：需匿名化、聚合处理
- 社交媒体签到：
 - 示例：微博、Twitter 带地理位置标签
 - 数据价值：热点区域发现、舆情监控
- 公共监控视频：
 - 技术：人脸识别、行为模式分析



公共监控视频中的人脸检测

被动感知：隐式数据采集

- 手机信令数据：
 - 来源：基站切换记录（时间、位置区码）
 - 应用：人流密度分析、通勤模式识别
 - 隐私保护：需匿名化、聚合处理
- 社交媒体签到：
 - 示例：微博、Twitter 带地理位置标签
 - 数据价值：热点区域发现、舆情监控
- 公共监控视频：
 - 技术：人脸识别、行为模式分析
 - 伦理要求：需符合相关法律法规



公共监控视频中的人脸检测

- 参与式感知应用:

- 参与式感知应用：
 - 环保监测：用户上报空气质量照片（如“蔚蓝地图”）



照片（如“蔚蓝地图”）
 交通拥堵信息

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- **参与式感知应用：**
 - 环保监测：用户上报空气质量照片（如“蔚蓝地图”）
 - 交通报告：Waze 用户主动提交拥堵信息
 - **数据特点：主观性强、需验证机制**

主动感知：用户参与式采集

- **参与式感知应用：**
 - 环保监测：用户上报空气质量照片（如“蔚蓝地图”）
 - 交通报告：Waze 用户主动提交拥堵信息
 - 数据特点：主观性强、需验证机制
- **智能设备交互：**

主动感知：用户参与式采集

- **参与式感知应用：**
 - 环保监测：用户上报空气质量照片（如“蔚蓝地图”）
 - 交通报告：Waze 用户主动提交拥堵信息
 - 数据特点：主观性强、需验证机制
- **智能设备交互：**
 - 示例：智能空调感知用户位置调节风向

照片（如“蔚蓝地图”）
交通拥堵信息
机制

调节风向
识别

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

主动感知：用户参与式采集

- **参与式感知应用：**
 - 环保监测：用户上传空气质量照片（如“蔚蓝地图”）
 - 交通报告：Waze 用户主动提交拥堵信息
 - 数据特点：主观性强、需验证机制
- **智能设备交互：**
 - 示例：智能空调感知用户位置调节风向
 - 技术：红外传感器、语音指令识别
- **众包地理信息：**

主动感知：用户参与式采集

- 参与式感知应用：
 - 环保监测：用户上报空气质量照片（如“蔚蓝地图”）
 - 交通报告：Waze 用户主动提交拥堵信息
 - 数据特点：主观性强、需验证机制
- 智能设备交互：
 - 示例：智能空调感知用户位置调节风向
 - 技术：红外传感器、语音指令识别
- 众包地理信息：
 - OpenStreetMap：用户共同绘制地图

主动感知：用户参与式采集

- 参与式感知应用：
 - 环保监测：用户上传空气质量照片（如“蔚蓝地图”）
 - 交通报告：Waze 用户主动提交拥堵信息
 - 数据特点：主观性强、需验证机制
- 智能设备交互：
 - 示例：智能空调感知用户位置调节风向
 - 技术：红外传感器、语音指令识别
- 众包地理信息：
 - OpenStreetMap：用户共同绘制地图
 - 质量控制：历史贡献度加权

- **核心思想：融合固定与移动感知、物与人的数据**

- **核心思想**：融合固定与移动感知、物与人的数据
- **系统架构**：

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

- 智能采样:

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

AI 赋能的采集优化

- 智能采样：
 - 强化学习动态调整传感器频率

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

AI 赋能的采集优化

- 智能采样：
 - 强化学习动态调整传感器频率
 - 示例：根据交通流量自适应调整GPS记录间隔

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- **智能采样：**
 - 强化学习动态调整传感器频率
 - 示例：根据交通流量自适应调整GPS 记录间隔
- **自动标注：**

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- 智能采样：
 - 强化学习动态调整传感器频率
 - 示例：根据交通流量自适应调整GPS记录间隔
- 自动标注：
 - 深度学习识别遥感影像地物（如房屋、道路）

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- 智能采样：
 - 强化学习动态调整传感器频率
 - 示例：根据交通流量自适应调整GPS 记录间隔
- 自动标注：
 - 深度学习识别遥感影像地物（如房屋、道路）
 - 基于配送轨迹的内部道路恢复

14 / 105

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- 智能采样：
 - 强化学习动态调整传感器频率
 - 示例：根据交通流量自适应调整GPS 记录间隔
- 自动标注：
 - 深度学习识别遥感影像地物（如房屋、道路）
 - 基于配送轨迹的内部道路恢复
 - 工具：Labellmg、CVAT
- 生成式填充：

多源协同采集技术

- 核心思想：融合固定与移动感知、物与人的数据
- 系统架构：
 - 数据层：分布式存储（HDFS、TiDB）
 - 平台层：任务调度、质量监控
 - 应用层：Web 端与移动端协同
- 案例：农村土地监测
 - 固定感知：卫星影像（月度更新）
 - 移动感知：无人机季度巡检
 - 主动感知：村民 APP 上报变更
 - 成果：违规用地识别准确率 91.3%^a

^a数据标注优秀案例集之六 | 农村集体土地高质量时空数据集多源协同

AI 赋能的采集优化

- 智能采样：
 - 强化学习动态调整传感器频率
 - 示例：根据交通流量自适应调整GPS 记录间隔
- 自动标注：
 - 深度学习识别遥感影像地物（如房屋、道路）
 - 基于配送轨迹的内部道路恢复
 - 工具：Labellmg、CVAT
- 生成式填充：
 - GAN 补全缺失数据（如云遮挡的卫星影像）

以政务数据为主的结构化数据
以视频、文件、文本为主的非结构化数据
以轨迹为主的大规模时空数据存储技术

② 时空数据的存储

以政务数据为主的结构化数据

以视频、文件、文本为主的非结构化数据

以轨迹为主的大规模时空数据存储技术

数据特点

- **结构化程度高**: 政务表格具有固定的字段和格式

数据特点

- **结构化程度高**: 政务表格具有固定的字段和格式
- **业务关联性强**: 与具体政务业务场景紧密相关

数据特点

- 结构化程度高: 政务表格具有固定的字段和格式
- 业务关联性强: 与具体政务业务场景紧密相关
- 多源异构: 来自不同部门, 标准不统一

政务表单数据特征

数据特点

- 结构化程度高: 政务表格具有固定的字段和格式
- 业务关联性强: 与具体政务业务场景紧密相关
- 多源异构: 来自不同部门, 标准不统一
- 价值密度高: 包含重要的城市运行管理信息

政务表单数据特征

数据特点

- **结构化程度高**: 政务表格具有固定的字段和格式
- **业务关联性强**: 与具体政务业务场景紧密相关
- **多源异构**: 来自不同部门, 标准不统一
- **价值密度高**: 包含重要的城市运行管理信息

典型政务数据源

- 人口户籍管理表格
- 企业工商注册信息
- 行政审批表单
- 行政执法记录
- 公共服务办理记录

现状分析

- ## ● 数字中国战略的必备路径



现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”

- 数学
本系

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

- 城市应用场景多变

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

- 城市应用场景多变
 - 需要全面、准确的覆盖个性化的城市业务场景

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

- 城市应用场景多变
 - 需要全面、准确的覆盖个性化的城市业务场景
 - 如何保证不同业务上的特定描述和数据要素体系规范化描述的共存

建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

- 城市应用场景多变
 - 需要全面、准确的覆盖个性化的城市业务场景
 - 如何保证不同业务上的特定描述和数据要素体系规范化描述的共存
- 数据要素体系构建难

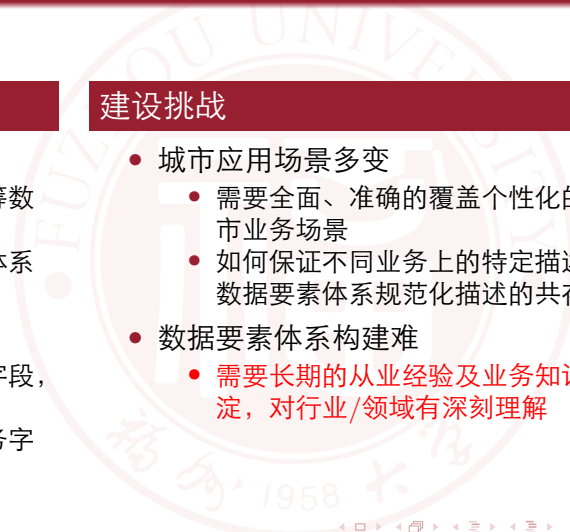
建立城市知识体系的背景

现状分析

- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

- 城市应用场景多变
 - 需要全面、准确的覆盖个性化的城市业务场景
 - 如何保证不同业务上的特定描述和数据要素体系规范化描述的共存
- 数据要素体系构建难
 - 需要长期的从业经验及业务知识沉淀，对行业/领域有深刻理解



现状分析

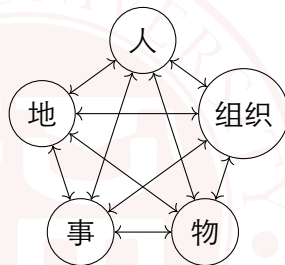
- 数字中国战略的必备路径
 - 国家层面成立国家数据局，统筹数据资源整合共享和开发利用
 - 夯实数字基础设施和数据资源体系“两大基础”
- 各地数据治理如火如荼
 - 后端研发基于业务需求创建表字段，基于个人经验的字段命名
 - 数据研发需理解不同标准的业务字段，存在大量的沟通成本

建设挑战

- 城市应用场景多变
 - 需要全面、准确的覆盖个性化的城市业务场景
 - 如何保证不同业务上的特定描述和数据要素体系规范化描述的共存
- 数据要素体系构建难
 - 需要长期的从业经验及业务知识沉淀，对行业/领域有深刻理解
 - 需要高度凝练及抽象能力，将过往碎片化经验进行提炼

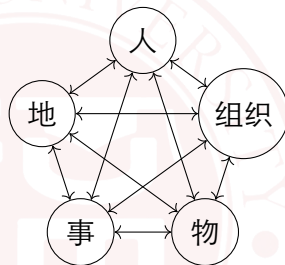
Journal of Management Education 36(7) 809–826

- **实体识别**: 识别表单中的人、地、事、物、组织五类实体



基于城市知识体系的数据标准化梳理

- **实体识别**: 识别表单中的人、地、事、物、组织五类实体
- **属性映射**: 将表单字段映射到标准属性体系



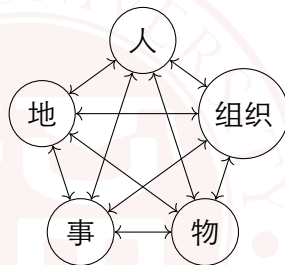
-

Journal of Management Inquiry 20(6) 798–814

-

基于城市知识体系的数据标准化梳理

- **实体识别**: 识别表单中的人、地、事、物、组织五类实体
- **属性映射**: 将表单字段映射到标准属性体系
- **关系建立**: 建立实体间的关联关系
- **元数据统一**: 统一字段命名规范和数据类型



标准化优势

- 减少数据理解成本
- 提高数据互操作性
- 支持跨部门数据共享

实体与相关属性

- 人：姓名、性别、身高、体重、年龄、出生日期、民族、籍贯、户籍所在地、证件号码、学历、政治面貌、联系方式

实体与相关属性

- 人：姓名、性别、身高、体重、年龄、出生日期、民族、籍贯、户籍所在地、证件号码、学历、政治面貌、联系方式
- 地：名称、功能、位置、地址、尺寸、容量、地形等

实体与相关属性

- 人：姓名、性别、身高、体重、年龄、出生日期、民族、籍贯、户籍所在地、证件号码、学历、政治面貌、联系方式
- 地：名称、功能、位置、地址、尺寸、容量、地形等
- 事：时间、主体、地点、类型、描述、结果、原因等

实体与相关属性

- 人：姓名、性别、身高、体重、年龄、出生日期、民族、籍贯、户籍所在地、证件号码、学历、政治面貌、联系方式
- 地：名称、功能、位置、地址、尺寸、容量、地形等
- 事：时间、主体、地点、类型、描述、结果、原因等
- 物：名称、功能、型号、形状、尺寸、颜色、重量、材质、使用期限等。

城市知识体系示例

实体与相关属性

- 人：姓名、性别、身高、体重、年龄、出生日期、民族、籍贯、户籍所在地、证件号码、学历、政治面貌、联系方式
- 地：名称、功能、位置、地址、尺寸、容量、地形等
- 事：时间、主体、地点、类型、描述、结果、原因等
- 物：名称、功能、型号、形状、尺寸、颜色、重量、材质、使用期限等。
- 组织：名称、类型、职责/经营范围、人员规模、曾用名、电话、邮箱、网址、地址、组织机构代码、统一社会信用代码、成立日期等

实体类型之间的关系

- 同一类型实体之间的关系



实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等
- 不同类型实体之间的关系

城市知识体系示例

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等
- 不同类型实体之间的关系
 - 人-地关系：常驻、访问、途经、建设、拥有、出让、管理、维护等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等
- 不同类型实体之间的关系
 - 人-地关系：常驻、访问、途经、建设、拥有、出让、管理、维护等
 - 人-物关系：如设计、制造、拥有、使用、出让、管理、维护等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等
- 不同类型实体之间的关系
 - 人-地关系：常驻、访问、途经、建设、拥有、出让、管理、维护等
 - 人-物关系：如设计、制造、拥有、使用、出让、管理、维护等
 - 人-组织关系：如创立、管理、从属、被授予、服务等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等
- 不同类型实体之间的关系
 - 人-地关系：常驻、访问、途经、建设、拥有、出让、管理、维护等
 - 人-物关系：如设计、制造、拥有、使用、出让、管理、维护等
 - 人-组织关系：如创立、管理、从属、被授予、服务等
 - 组织-地关系：如驻扎、建设、拥有、出让、管理、维护等

实体类型之间的关系

- 同一类型实体之间的关系
 - 人-人关系：如婚姻、服务、冲突等
 - 物-物关系：如从属、连接、空间、承载等
 - 事-事关系：如因果、关联、时间等
 - 组织-组织关系：如从属、指导、协作、竞争等
 - 地-地关系：如邻接、从属、相对空间等
- 不同类型实体之间的关系
 - 人-地关系：常驻、访问、途经、建设、拥有、出让、管理、维护等
 - 人-物关系：如设计、制造、拥有、使用、出让、管理、维护等
 - 人-组织关系：如创立、管理、从属、被授予、服务等
 - 组织-地关系：如驻扎、建设、拥有、出让、管理、维护等
 - 地-事关系：如承载、参与、被影响等
 - 事-组织关系：如发起、参与、管理等

主题库特征

- 业务导向组织
- 聚合计算结果
- 优化查询性能
- 支持分析应用

基础库特征

- 原始数据保持
- 最小粒度存储
- 长期归档保存
- 支持数据追溯

主题库特征

- 业务导向组织
- 聚合计算结果
- 优化查询性能
- 支持分析应用

② 时空数据的存储

以政务数据为主的结构化数据

以视频、文件、文本为主的非结构化数据

以轨迹为主的大规模时空数据存储技术

定义

非结构化数据是指不遵循固定模式或不具有预定义数据模型的数据，其内容和结构通常由数据的自然形式决定。

主要特征:

- 无固定格式和模式
- 数据量巨大且增长快速
- 类型多样且结构复杂
- 价值密度相对较低

非结构化数据定义与特征

定义

非结构化数据是指不遵循固定模式或不具有预定义数据模型的数据，其内容和结构通常由数据的自然形式决定。

主要特征:

- 无固定格式和模式
- 数据量巨大且增长快速
- 类型多样且结构复杂
- 价值密度相对较低

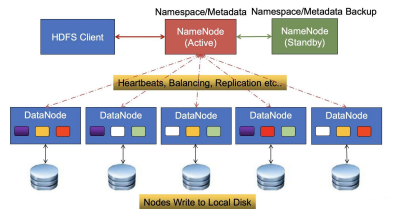
与结构化数据对比:

非结构化数据	结构化数据
无固定模式	预定义模式
文本、视频、图像等	数据库表格形式
占企业数据 80% 以上	占比较小
分析复杂度高	分析工具成熟

26 / 105

主流分布式文件系统

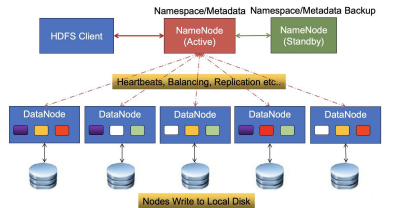
- **Google GFS**: Google 开发的分布式文件系统
- **Hadoop HDFS**: GFS 的开源实现, Hadoop 生态系统核心存储组件
- **Ceph**: 高性能、高可靠的统一分布式存储。可作为 Hadoop 集群的底层存储后端。分布式元数据服务器 (MDS)。



分布式文件系统架构

主流分布式文件系统

- **Google GFS**: Google 开发的分布式文件系统
- **Hadoop HDFS**: GFS 的开源实现, Hadoop 生态系统核心存储组件
- **Ceph**: 高性能、高可靠的统一分布式存储。可作为 Hadoop 集群的底层存储后端。分布式元数据服务器 (MDS)。
- **FastDFS**: 轻量级, 部署简单, 高效处理中小文件, 适合中小规模场景。

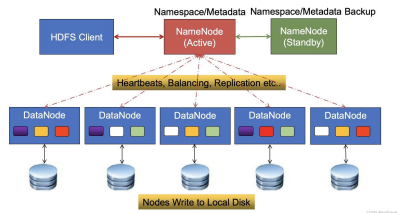


分布式文件系统架构

分布式文件系统存储

主流分布式文件系统

- **Google GFS**: Google 开发的分布式文件系统
- **Hadoop HDFS**: GFS 的开源实现, Hadoop 生态系统核心存储组件
- **Ceph**: 高性能、高可靠的统一分布式存储。可作为 Hadoop 集群的底层存储后端。分布式元数据服务器 (MDS)。
- **FastDFS**: 轻量级, 部署简单, 高效处理中小文件, 适合中小规模场景。



分布式文件系统架构

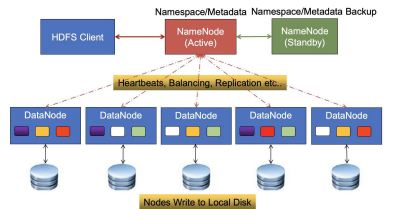
共同的优势

- 高吞吐量数据访问
- 数据备份和容错机制

分布式文件系统存储

主流分布式文件系统

- **Google GFS**: Google 开发的分布式文件系统
- **Hadoop HDFS**: GFS 的开源实现, Hadoop 生态系统核心存储组件
- **Ceph**: 高性能、高可靠的统一分布式存储。可作为 Hadoop 集群的底层存储后端。分布式元数据服务器 (MDS)。
- **FastDFS**: 轻量级, 部署简单, 高效处理中小文件, 适合中小规模场景。



分布式文件系统架构

共同的优势

- 高吞吐量数据访问
- 数据备份和容错机制
- 适合海量非结构化数据存储

NoSQL 数据库是一类非关系型数据库，设计用于处理大规模分布式数据存储，支持灵活的数据模型，适合存储非结构化和半结构化数据。

NoSQL 数据库

定义

NoSQL 数据库是一类非关系型数据库，设计用于处理大规模分布式数据存储，支持灵活的数据模型，适合存储非结构化和半结构化数据。

NoSQL 数据库类型

- 键值数据库：Redis、Memcached

NoSQL 数据库是一类非关系型数据库，设计用于处理大规模分布式数据存储，支持灵活的数据模型，适合存储非结构化和半结构化数据。

NoSQL 数据库类型

- 键值数据库: Redis、Memcached
- 文档数据库: MongoDB、CouchDB
- 列族数据库: Cassandra、HBase
- 图数据库: Neo4j、JanusGraph

定义

面向对象存储（Object-based Storage）是一种将数据、可扩展的元数据以及全局唯一标识符封装为一个独立对象，并通过扁平化结构和 RESTful API 进行访问的存储架构，专为高效管理海量非结构化数据而设计。

发展历史

- 面向对象存储主要源于两个方面的驱动因素：数据量的爆炸式增长和非结构化数据比例的显著提升。传统的块存储（如 DAS、SAN）和文件存储（NAS）在可扩展性（通常限于 PB 级）和管理效率上显得力不从心。

定义

面向对象存储（Object-based Storage）是一种将数据、可扩展的元数据以及全局唯一标识符封装为一个独立对象，并通过扁平化结构和 RESTful API 进行访问的存储架构，专为高效管理海量非结构化数据而设计。

发展历史

- 面向对象存储主要源于两个方面的驱动因素：数据量的爆炸式增长和非结构化数据比例的显著提升。传统的块存储（如 DAS、SAN）和文件存储（NAS）在可扩展性（通常限于 PB 级）和管理效率上显得力不从心。
- 2006 年 Amazon S3（Simple Storage Service）服务的推出，它将对象存储作为一种云服务带入主流视野，并基本确立了该领域的事实标准。

定义

面向对象存储（Object-based Storage）是一种将数据、可扩展的元数据以及全局唯一标识符封装为一个独立对象，并通过扁平化结构和 RESTful API 进行访问的存储架构，专为高效管理海量非结构化数据而设计。

发展历史

- 面向对象存储主要源于两个方面的驱动因素：数据量的爆炸式增长和非结构化数据比例的显著提升。传统的块存储（如 DAS、SAN）和文件存储（NAS）在可扩展性（通常限于 PB 级）和管理效率上显得力不从心。
- 2006 年 Amazon S3（Simple Storage Service）服务的推出，它将对象存储作为一种云服务带入主流视野，并基本确立了该领域的事实标准。
- 主要以云服务的形式提供。国内外主流云厂商都推出了自己的对象存储服务，虽然名称各异，但技术核心和 API（如 S3 兼容接口）基本一致。

采用扁平的命名空间和分布式集群架构，理论容量，满足海量数据存储需求。

- 近乎无限的扩展能力：对象存储采用扁平的命名空间和分布式集群架构，理论上可以轻松扩展至 EB 级甚至更大容量，满足海量数据存储需求。

采用扁平的命名空间和分布式集群架构，理论容量，满足海量数据存储需求。

多节点冗余存储（通常至少 3 个副本或使用纠删码）达 99.999999999%（11 个 9）的数据耐久性，

- 近乎无限的扩展能力：对象存储采用扁平的命名空间和分布式集群架构，理论上可以轻松扩展至 EB 级甚至更大容量，满足海量数据存储需求。
- 高可靠性与耐久性：数据会通过多节点冗余存储（通常至少 3 个副本或使用纠删码技术），服务商通常承诺高达 99.999999999%（11 个 9）的数据耐久性，这意味着数据丢失的概率极低。

采用扁平的命名空间和分布式集群架构，理论容量，满足海量数据存储需求。

多节点冗余存储（通常至少 3 个副本或使用纠删码）达 99.999999999%（11 个 9）的数据耐久性，

RESTful API（如 PUT, GET, DELETE）进行自定义元数据，便于数据的分类、检索和管理。类似于使用网盘，非常方便。

- 近乎无限的扩展能力：对象存储采用扁平的命名空间和分布式集群架构，理论上可以轻松扩展至 EB 级甚至更大容量，满足海量数据存储需求。
- 高可靠性与耐久性：数据会通过多节点冗余存储（通常至少 3 个副本或使用纠删码技术），服务商通常承诺高达 99.999999999%（11 个 9）的数据耐久性，这意味着数据丢失的概率极低。
- 易于使用与管理：它通过简单的 RESTful API（如 PUT, GET, DELETE）进行操作，并可为每个对象附加丰富的自定义元数据，便于数据的分类、检索和管理。对于用户而言，存取数据类似于使用网盘，非常方便。

采用扁平的命名空间和分布式集
大容量，满足海量数据存储需求
多节点冗余存储（通常至少 3 个
达 99.999999999%（11 个 9）的数
RESTful API（如 PUT, GET, D
的自定义元数据，便于数据的分
类似于使用网盘，非常方便。
常提供标准、低频访问、归档等
重要性选择不同层级的服务，从
缩，用户无需提前规划容量或进

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

非结构化数据索引技术

全文检索技术

- **倒排索引**：词条-文档映射关系
- **正排索引**：文档-词条顺序关系
- **搜索引擎**：Elasticsearch、Solr、Lucene

图像索引技术

- **SIFT 特征提取**: 尺度不变特征变换
- 颜色直方图索引
- 形状特征索引
- 纹理特征分析

音频索引技术

- 语音识别转换
- 声纹特征提取
- 音频指纹技术
- 音乐信息检索

非结构化数据索引技术

全文检索技术

- **倒排索引**：词条-文档映射关系
- **正排索引**：文档-词条顺序关系
- **搜索引擎**：Elasticsearch、Solr、Lucene

图像索引技术

- **SIFT 特征提取**: 尺度不变特征变换
- 颜色直方图索引
- 形状特征索引
- 纹理特征分析

音频索引技术

- 语音识别转换
- 声纹特征提取
- 音频指纹技术
- 音乐信息检索

视频索引技术

- 关键帧提取
- 场景分割识别
- 运动特征分析
- 音频特征索引

② 时空数据的存储

以视频、文件、文本为主的非结构化数据

以轨迹为主的大规模时空数据存储技术

K-D 树

GeoHash 编码

时空数据库简介

PostGIS 详解

GeoMesa

③ 时空数据的处理

K-D 树的构建步骤

① 选择切分维度 (Split Dimension):



K-D 树的构建步骤

① 选择切分维度 (Split Dimension):

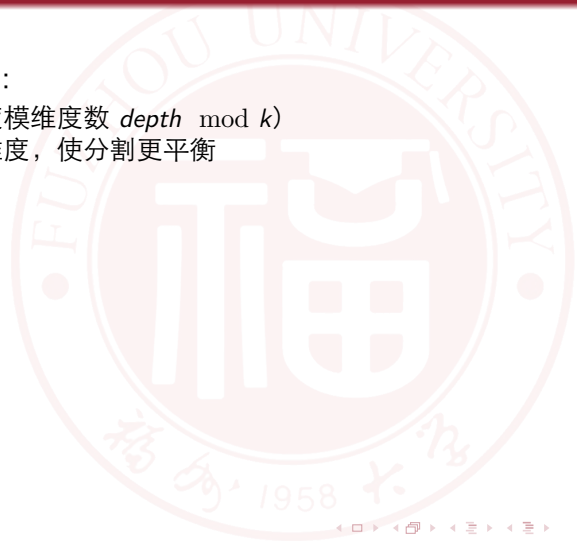
- 常见策略：轮流选择（如深度模维度数 $depth \bmod k$ ）



K-D 树的构建步骤

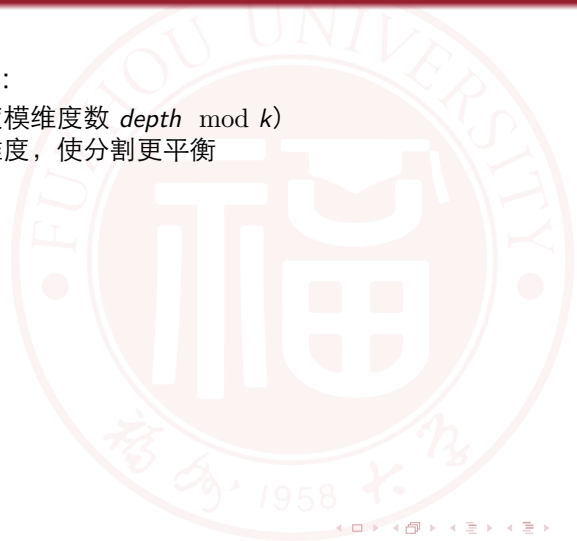
① 选择切分维度 (Split Dimension):

- 常见策略：轮流选择（如深度模维度数 $depth \bmod k$ ）
- 优化策略：选择方差最大的维度，使分割更平衡



K-D 树的构建步骤

- ① 选择切分维度 (Split Dimension):
 - 常见策略：轮流选择（如深度模维度数 $depth \bmod k$ ）
 - 优化策略：选择方差最大的维度，使分割更平衡
- ② 选择切分点 (Split Point):



K-D 树的构建步骤

① 选择切分维度 (Split Dimension):

- 常见策略：轮流选择（如深度模维度数 $depth \bmod k$ ）
- 优化策略：选择方差最大的维度，使分割更平衡

② 选择切分点 (Split Point):

- 通常取当前维度数据的中位数作为切分点，确保左右子树节点数平衡

① 选择切分维度 (Split Dimension):

- 常见策略: 轮流选择 (如深度模维度数 $depth \bmod k$)
- 优化策略: 选择方差最大的维度, 使分割更平衡

② 选择切分点 (Split Point):

- 通常取当前维度数据的中位数作为切分点，确保左右子树节点数平衡

③ 递归构建：

K-D 树的构建步骤

① 选择切分维度 (Split Dimension):

- 常见策略: 轮流选择 (如深度模维度数 $depth \bmod k$)
- 优化策略: 选择方差最大的维度, 使分割更平衡

② 选择切分点 (Split Point):

- 通常取当前维度数据的中位数作为切分点，确保左右子树节点数平衡

③ 递归构建：

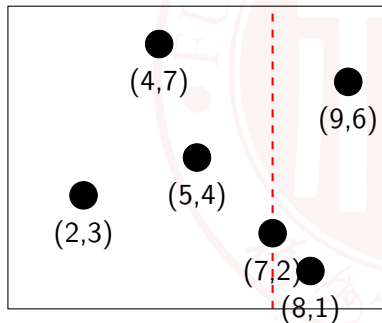
- 将数据按切分维度分为两部分，分别递归构建左子树和右子树

构建示例：二维数据点

例

以二维点集为例： $\{(2,3), (5,4), (9,6), (4,7), (8,1), (7,2)\}$

根节点: $(7,2)$, $\text{split}=x$



最近邻搜索 (Nearest Neighbor Search)

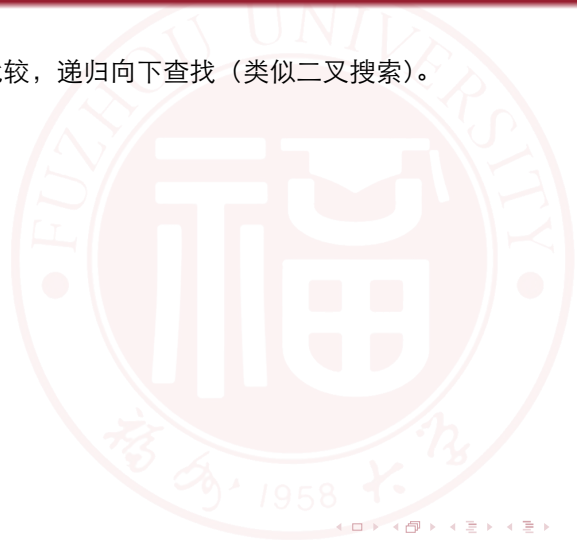
- 基本思想:



最近邻搜索 (Nearest Neighbor Search)

- 基本思想:

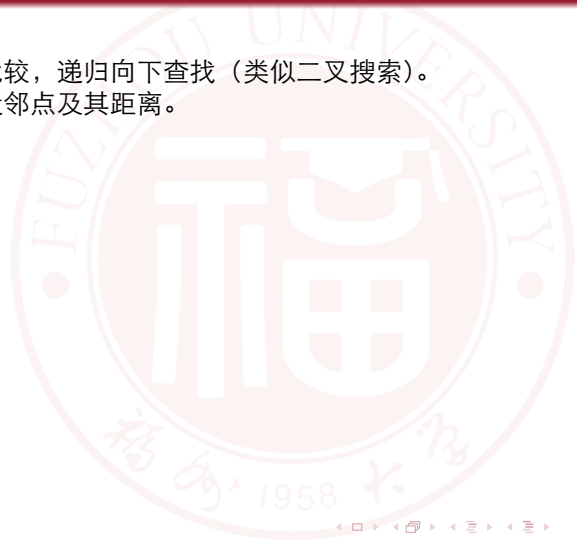
- ① 从根节点开始, 按相应维度比较, 递归向下查找 (类似二叉搜索)。



最近邻搜索 (Nearest Neighbor Search)

- 基本思想:

- ① 从根节点开始, 按相应维度比较, 递归向下查找 (类似二叉搜索)。
- ② 到达叶节点后, 记录当前最近邻点及其距离。



较，递归向下查找（类似二叉搜索）。
邻点及其距离。
子树（若以查询点为圆心，当前最短距离为半径的
查另一子树）。

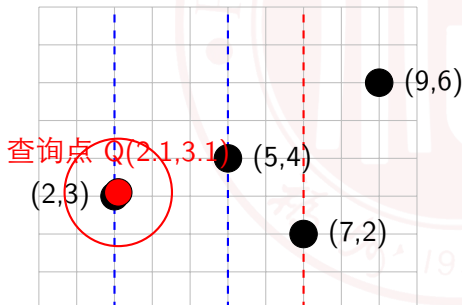
- 1 从根节点开始，按相应维度比较，递归向下查找（类似二叉搜索）。
- 2 到达叶节点后，记录当前最近邻点及其距离。
- 3 回溯：判断是否需要检查另一子树（若以查询点为圆心，当前最短距离为半径的圆与分割超平面相交，则需检查另一子树）。

最近邻搜索 (Nearest Neighbor Search)

- 基本思想:

- ① 从根节点开始, 按相应维度比较, 递归向下查找 (类似二叉搜索)。
- ② 到达叶节点后, 记录当前最近邻点及其距离。
- ③ 回溯: 判断是否需要检查另一子树 (若以查询点为圆心, 当前最短距离为半径的圆与分割超平面相交, 则需检查另一子树)。

回溯判断



节点插入与删除

插入操作

- 类似二叉搜索树：从根开始，根据相应维度比较，递归找到插入位置。
- 可能破坏树平衡性，多次插入后需重新平衡。

- 可能破坏树平衡性，多次插入后需重新平衡。

删除操作

- 若待删除节点是叶节点，直接删除。
- 若只有一棵子树，用子树替代。
- 若有两棵子树，需找到其右子树中当前划分维度的最小值节点（或左子树中最大值节点）来替代被删节点，并递归删除那个节点。

- 若只有一棵子树，用子树替代。

- 若有两棵子树，需找到其右子树中当前划分维度的最小值节点（或左子树中最大值节点）来替代被删节点，并递归删除那个节点。

节点插入与删除

插入操作

- 类似二叉搜索树：从根开始，根据相应维度比较，递归找到插入位置。
- 可能破坏树平衡性，多次插入后需重新平衡。

删除操作

- 若待删除节点是叶节点，直接删除。
- 若只有一棵子树，用子树替代。
- 若有两棵子树，需找到其右子树中当前划分维度的最小值节点（或左子树中最大值节点）来替代被删节点，并递归删除那个节点。

注意：

频繁的插入和删除可能显著破坏 K-D 树的平衡性，影响查询效率。在实际应用中，有时倾向于重建子树或整棵树。

复杂度分析

表 1: K-D 树各操作时间复杂度

操作	平均情况	最坏情况
构建	$O(n \log n)$	$O(n^2)$
最近邻搜索	$O(\log n)$	$O(n)$
范围查询	$O(n^{1-1/k} + m)^*$	$O(n)$
插入节点	$O(\log n)$	$O(n)$
删除节点	$O(\log n)$	$O(n)$

* 其中, k 是数据维度, m 是查询范围内点的数量。范围查询的平均复杂度与维度密切相关。

影响效率的关键因素:

- 树的平衡性。
- 维度灾难 (Curse of Dimensionality): 当维度 k 很高时 (例如 $k > 20$), 查询效率可能趋近于线性扫描 $O(n)$ 。要求数据规模 $N \gg 2^D$ 才能保持高效。

预备知识: GeoHash

空间数据检索的挑战

- 传统空间索引复杂度高: R 树、四叉树等实现复杂
- 数据库查询效率低: 经纬度范围查询性能瓶颈
- 分布式系统支持弱: 空间数据分片困难
- 缓存优化难度大: 地理位置相关性难以利用

核心需求

急需一种将二维地理坐标转换为一维字符串的编码方法，实现：

- 快速邻近搜索
- 简单数据结构
- 良好可扩展性

定义

GeoHash 是一种将二维地理坐标（经纬度）编码为一维字符串的空间索引算法，具有以下特性：

核心特性:

- 层次化空间划分
- 前缀匹配特性
- 自包含位置信息
- 字符串表示形式

应用场景

- 地理位置搜索
- 附近点查询
- 热力图生成
- 轨迹数据索引

基础思想：空间递归分割

- 将地球表面视为一个二维平面
- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$
- 通过二进制分割实现层次化编码

Level 1: 4 个网格

00	01 0100	10	11

GeoHash 层次化空间分割示意图

编码计算步骤

步骤 1: 二进制编码生成

- ① 初始化经度范围: $\text{lon_min} = -180, \text{lon_max} = 180$
- ② 初始化纬度范围: $\text{lat_min} = -90, \text{lat_max} = 90$
- ③ 循环指定精度次数 (通常 12-15 次)
- ④ 每次循环计算中间值, 判断位置

经度编码示例 (116.3912°E)

- 第一次: $(-180, 180) \rightarrow$ 中间值 0 , $116.39 > 0 \rightarrow$ 编码 1
- 第二次: $(0, 180) \rightarrow$ 中间值 90 , $116.39 > 90 \rightarrow$ 编码 1
- 第三次: $(90, 180) \rightarrow$ 中间值 135 , $116.39 < 135 \rightarrow$ 编码 0
- 继续直到达到所需精度...

Base32 编码转换

二进制到字符串的映射

将二进制序列每 5 位一组，映射到 Base32 字符集：

Base32 编码映射表

十进制	二进制	字符	十进制	二进制	字符	十进制	二进制	字符	十进制	二进制	字符
0	00000	0	8	01000	8	16	10000	h	24	11000	s
1	00001	1	9	01001	9	17	10001	j	25	11001	t
2	00010	2	10	01010	b	18	10010	k	26	11010	u
3	00011	3	11	01011	c	19	10011	m	27	11011	v
4	00100	4	12	01100	d	20	10100	n	28	11100	w
5	00101	5	13	01101	e	21	10101	p	29	11101	x
6	00110	6	14	01110	f	22	10110	q	30	11110	y
7	00111	7	15	01111	g	23	10111	r	31	11111	z

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围：

- 经度范围：[-180, 180]
- 纬度范围：[-90, 90]



完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围：

- 经度范围：[-180, 180]
- 纬度范围：[-90, 90]

② 二进制编码过程（以经度为例）：



完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

- $116.3912 \in [-180, 180]$

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

- $116.3912 \in [-180, 180]$
- 中点: 0, $116.3912 > 0 \rightarrow 1$



完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

- $116.3912 \in [-180, 180]$
- 中点: 0, $116.3912 > 0 \rightarrow 1$
- 新区间: $[0, 180]$, 中点: 90, $116.3912 > 90 \rightarrow 1$

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

- $116.3912 \in [-180, 180]$
- 中点: 0, $116.3912 > 0 \rightarrow 1$
- 新区间: $[0, 180]$, 中点: 90, $116.3912 > 90 \rightarrow 1$
- 新区间: $[90, 180]$, 中点: 135, $116.3912 < 135 \rightarrow 0$

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

- $116.3912 \in [-180, 180]$
- 中点: 0, $116.3912 > 0 \rightarrow 1$
- 新区间: $[0, 180]$, 中点: 90, $116.3912 > 90 \rightarrow 1$
- 新区间: $[90, 180]$, 中点: 135, $116.3912 < 135 \rightarrow 0$
- 新区间: $[90, 135]$, 中点: 112.5, $116.3912 > 112.5 \rightarrow 1$

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

- $116.3912 \in [-180, 180]$
- 中点: 0, $116.3912 > 0 \rightarrow 1$
- 新区间: $[0, 180]$, 中点: 90, $116.3912 > 90 \rightarrow 1$
- 新区间: $[90, 180]$, 中点: 135, $116.3912 < 135 \rightarrow 0$
- 新区间: $[90, 135]$, 中点: 112.5, $116.3912 > 112.5 \rightarrow 1$
- 继续分割直到达到精度...

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

③ 最终二进制表示:

- 经度二进制: **11010 01011 00010 00101...**
- 纬度二进制: **10111 00011 00010 10000...**

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

③ 最终二进制表示:

- 经度二进制: **11010 01011 00010 00101...**
- 纬度二进制: **10111 00011 00010 10000...**

④ 位交错合并:

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

③ 最终二进制表示:

- 经度二进制: **11010 01011 00010 00101...**
- 纬度二进制: **10111 00011 00010 10000...**

④ 位交错合并:

- 经度位: 1 1 0 1 0 0 1 0...

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

③ 最终二进制表示:

- 经度二进制: **11010 01011 00010 00101...**
- 纬度二进制: **10111 00011 00010 10000...**

④ 位交错合并:

- 经度位: 1 1 0 1 0 0 1 0...
- 纬度位: 1 0 1 1 1 0 0 0...

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围:

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程 (以经度为例):

③ 最终二进制表示:

- 经度二进制: **11010 01011 00010 00101...**
- 纬度二进制: **10111 00011 00010 10000...**

④ 位交错合并:

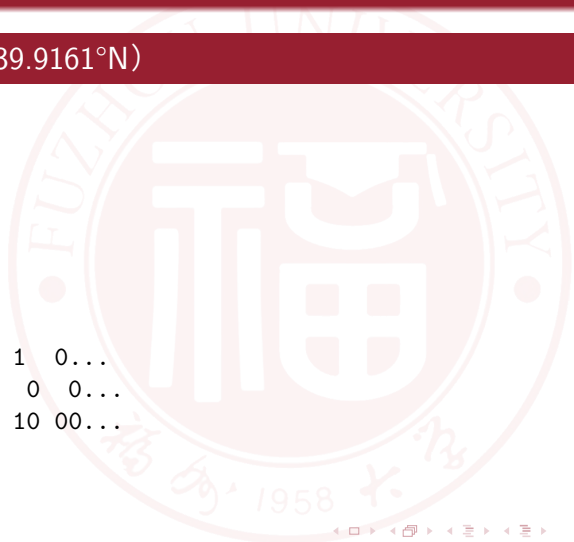
- 经度位: 1 1 0 1 0 0 1 0...
- 纬度位: 1 0 1 1 1 0 0 0...
- 交错后: **11 10 01 11 01 00 10 00...**

完整计算示例

北京故宫坐标编码 (116.3912°E, 39.9161°N)

- ① 初始化范围：
 - 经度范围：[-180, 180]
 - 纬度范围：[-90, 90]
- ② 二进制编码过程（以经度为例）：
- ③ 最终二进制表示：
- ④ 位交错合并：
 - 经度位：1 1 0 1 0 0 1 0...
 - 纬度位：1 0 1 1 1 0 0 0...
 - 交错后：11 10 01 11 01 00 10 00...

⑤ Base32 编码转换：



北京故宫坐标编码 (116.3912°E, 39.9161°N)

① 初始化范围：

- 经度范围: $[-180, 180]$
- 纬度范围: $[-90, 90]$

② 二进制编码过程（以经度为例）：

③ 最终二进制表示:

④ 位交错合并:

- 经度位: 1 1 0 1 0 0 1 0...
- 纬度位: 1 0 1 1 1 0 0 0...
- 交错后: 11 10 01 11 01 00 10 00...

5 Base32 编码转换:

- 5 位分组: 11100 11101 01101...

课堂练习

计算下列点的 GeoHash 编码：

- ① 31.2364° N, 121.5056° E（上海外滩）

课堂练习

计算下列点的 GeoHash 编码：

- ① 31.2364° N, 121.5056° E（上海外滩）
- ② $22^{\circ}32'36.5''$ N $113^{\circ}55'47''$ E（腾讯大厦）

课堂练习

计算下列点的 GeoHash 编码：

- ① 31.2364° N, 121.5056° E（上海外滩）
- ② $22^{\circ}32'36.5''$ N $113^{\circ}55'47''$ E（腾讯大厦）
- ③ 38.8977° N, 77.0365° W（白宫）

空间数据管理的挑战

- 数据特征复杂：空间数据具有空间位置、非结构化、复杂空间关系和海量特征
- 传统文件管理瓶颈：
 - Shapefile 等文件格式需要特殊软件读写
 - 并发用户可能导致数据损坏和性能下降
 - 复杂查询和分析能力有限
- 大数据时代新需求：物联网、轨迹记录等应用产生海量时空数据

解决方案演进路径

文件管理 → 文件与关系数据库混合 → 全关系型空间数据库 → 关系数据库 + 空间数据引擎 → 扩展对象关系型数据库

空间数据库解决方案概览

类型	代表产品	特点
扩展对象关系型数据库	PostGIS, Oracle Spatial	空间功能内嵌数据库内核，支持扩展SQL
关系数据库 + 空间数据引擎 分布式空间数据引擎	ArcSDE, SpatialWare GeoMesa, GeoWave	空间引擎作为中间件，数据库仅作存储 基于大数据生态，支持海量数据分布式处理

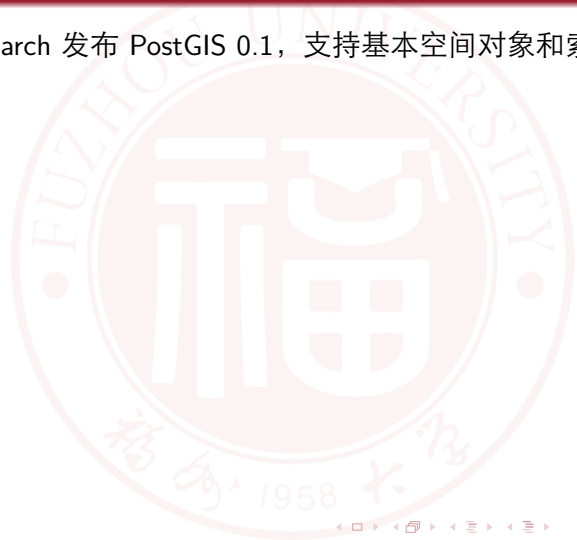
表 2: 空间数据库解决方案对比

技术路线分叉

PostGIS 代表单机强一致性路线，GeoMesa 代表分布式高可扩展性路线，两者适应不同应用场景

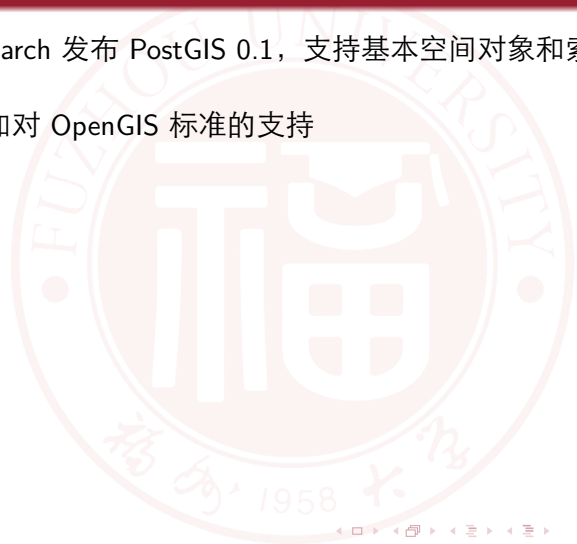
PostGIS 发展历程

- **2001 年 5 月**：Refractions Research 发布 PostGIS 0.1，支持基本空间对象和索引



PostGIS 发展历程

- **2001 年 5 月**：Refractions Research 发布 PostGIS 0.1，支持基本空间对象和索引
- **2001 年 7 月**：PostGIS 0.5 增加对 OpenGIS 标准的支持



PostGIS 发展历程

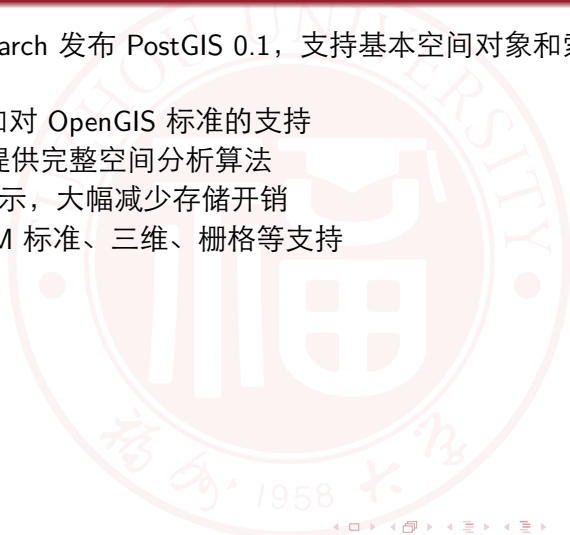
- **2001 年 5 月**：Refractions Research 发布 PostGIS 0.1，支持基本空间对象和索引
- **2001 年 7 月**：PostGIS 0.5 增加对 OpenGIS 标准的支持
- **关键转折点**：集成 GEOS 库，提供完整空间分析算法

PostGIS 发展历程

- **2001 年 5 月**：Refractions Research 发布 PostGIS 0.1，支持基本空间对象和索引
- **2001 年 7 月**：PostGIS 0.5 增加对 OpenGIS 标准的支持
- **关键转折点**：集成 GEOS 库，提供完整空间分析算法
- **版本 1.0**：采用轻量级元数据表示，大幅减少存储开销

PostGIS 发展历程

- **2001 年 5 月**：Refractions Research 发布 PostGIS 0.1，支持基本空间对象和索引
- **2001 年 7 月**：PostGIS 0.5 增加对 OpenGIS 标准的支持
- **关键转折点**：集成 GEOS 库，提供完整空间分析算法
- **版本 1.0**：采用轻量级元数据表示，大幅减少存储开销
- **持续演进**：增加对 ISO SQL/MM 标准、三维、栅格等支持



PostGIS 核心架构

架构层次：

- ① 存储层：PostgreSQL 表空间管理
- ② 数据类型：Geometry/Geography/Raster
- ③ 功能层：空间函数与操作符
- ④ 索引层：GiST/SP-GiST 索引
- ⑤ 接口层：SQL/ODBC/JDBC

核心依赖库

GEOS（几何算法）、Proj.4（投影转换）、GDAL（栅格处理）是 PostGIS 的三大支柱

PostGIS 功能特性

空间数据类型支持

- 基本几何类型：Point、LineString、Polygon 等
- 集合类型：MultiPoint、MultiPolygon 等
- 地理类型：Geography（球面计算）
- 栅格类型：Raster（PostGIS 2.0+）
- 三维类型：Z 坐标支持

核心功能

- 空间索引（GiST-based R 树）
- 空间谓词（ST_Contains、ST_Intersects 等）
- 空间操作（ST_Buffer、ST_Union 等）
- 空间分析（距离、面积、长度等）

PostGIS 空间索引技术

GiST 索引原理：

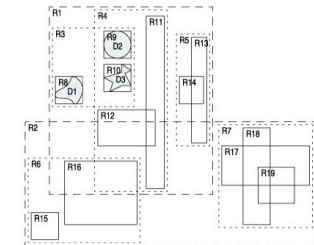
- 通用搜索树，支持任意数据分区策略
- 空间索引采用 R 树变体，基于对象 MBR 组织
- 支持 k-NN 搜索和空间谓词优化

空间查询优化示例

通过 GiST 索引快速过滤候选集，然后进行精确几何计算

CONCLUSIONS

- R8 R9 R10 R11 R12 R13 R14 R15 R16 R17 R18 R19 Leaf pages





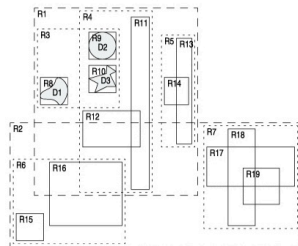
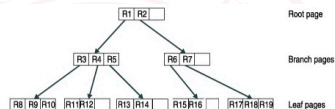
R 树简介

- R 树是一种用于多维空间数据（如地理坐标、几何对象）的平衡树索引结构。
- 由 Antonin Guttman 于 1984 年提出，是 B 树在多维空间下的扩展。
- 核心思想：使用最小边界矩形（MBR, Minimum Bounding Rectangle）来近似表示空间对象，并采用分层聚合的方式组织数据。
- 主要应用：地理信息系统（GIS）、空间数据库、计算机图形学中的碰撞检测和范围查询等。

- 由 Antonin Guttman 于 1984 年提出，是 B 树在多维空间下的扩展。

- 核心思想：使用最小边界矩形 (MBR, Minimum Bounding Rectangle) 来近似表示空间对象，并采用分层聚合的方式组织数据。

- 主要应用：地理信息系统 (GIS)、空间数据库、计算机图形学中的碰撞检测和范围查询等。



R 树结构示意图

核心数据结构与性质

节点结构

R 树的节点分为叶节点和非叶节点（内部节点）：

- 叶节点：存储数据条目，格式为 (MBR, Object Identifier)，直接指向磁盘上的 **空间对象数据**。
- 非叶节点：存储索引条目，格式为 (MBR, Child Pointer)，该 MBR 完全覆盖其所有子节点中条目的 MBR。

R 树的性质

- 平衡树：所有叶节点位于同一层。
- 节点容量：每个节点包含的条目数量在 m 至 M 之间 ($m \leq M/2$)，根节点除外（可少于 m 个条目）。
- **MBR 覆盖**：父节点条目的 MBR 必须完全覆盖其子节点所有条目的 MBR。

插入操作

插入新对象 E 及其 MBR 的主要步骤：

- ① **选择叶节点 (ChooseLeaf)**：从根节点开始，递归选择插入后其**MBR** 面积增加最小的子节点。若面积增加相同，则选择当前面积较小的节点。



插入操作

插入新对象 E 及其 MBR 的主要步骤：

- ① 选择叶节点 (**ChooseLeaf**)：从根节点开始，递归选择插入后其 **MBR 面积增加最小**的子节点。若面积增加相同，则选择当前面积较小的节点。
- ② 节点处理：



插入操作

插入新对象 E 及其 MBR 的主要步骤：

- ① 选择叶节点 (**ChooseLeaf**)：从根节点开始，递归选择插入后其 **MBR 面积增加最小** 的子节点。若面积增加相同，则选择当前面积较小的节点。
- ② 节点处理：
 - 若叶节点 L 有空间，直接插入 E 。

插入操作

插入新对象 E 及其 MBR 的主要步骤：

- ① 选择叶节点 (**ChooseLeaf**)：从根节点开始，递归选择插入后其 **MBR 面积增加最小** 的子节点。若面积增加相同，则选择当前面积较小的节点。
- ② 节点处理：
 - 若叶节点 L 有空间，直接插入 E 。
 - 若 L 已满（条目数 $> M$ ），则执行节点分裂 (**SplitNode**)。

插入操作

插入新对象 E 及其 MBR 的主要步骤：

- ① 选择叶节点 (**ChooseLeaf**)：从根节点开始，递归选择插入后其 **MBR 面积增加最小** 的子节点。若面积增加相同，则选择当前面积较小的节点。
- ② 节点处理：
 - 若叶节点 L 有空间，直接插入 E 。
 - 若 L 已满（条目数 $> M$ ），则执行节点分裂 (**SplitNode**)。
- ③ 调整树 (**AdjustTree**)：从 L 开始向上回溯至根节点，更新路径上受影响节点的 MBR。若分裂产生的节点 LL 导致父节点溢出，需递归分裂。

节点分裂

分裂目标

将溢出的节点 N 中的 $M+1$ 个条目划分为两个组，使得两个新节点的 MBR 尽可能不重叠，以减少查询时访问的路径数量。

常用分裂算法：二次方算法（Quadratic Split）概要

节点分裂

分裂目标

将溢出的节点 N 中的 $M+1$ 个条目划分为两个组，使得两个新节点的 MBR 尽可能不重叠，以减少查询时访问的路径数量。

常用分裂算法：二次方算法（Quadratic Split）概要

- ① 挑选种子：遍历所有条目对，选择使其合并后 MBR 面积增量最大的一对条目，作为两个新组的初始种子。

节点分裂

分裂目标

将溢出的节点 N 中的 $M+1$ 个条目划分为两个组，使得两个新节点的 MBR 尽可能不重叠，以减少查询时访问的路径数量。

常用分裂算法：二次方算法（Quadratic Split）概要

- ① 挑选种子：遍历所有条目对，选择使其合并后 MBR 面积增量最大的一对条目，作为两个新组的初始种子。
- ② 分配剩余条目：对于剩余每个条目，计算将其加入组 1 或组 2 后 MBR 的面积增量，将其分配到引起面积增量较小的那个组中。需确保分配后各组条目数满足最小值 m 。

查询操作

R 树的查询通常为范围查询（窗口查询），给定一个查询矩形 S ，找出所有与 S 相交的空间对象。

搜索算法（递归或迭代实现）



查询操作

R 树的查询通常为范围查询（窗口查询），给定一个查询矩形 S ，找出所有与 S 相交的空间对象。

搜索算法（递归或迭代实现）

- ① 若 T 为非叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与查询矩形 S 相交，则递归搜索 E 指向的子节点。

查询操作

R 树的查询通常为范围查询（窗口查询），给定一个查询矩形 S ，找出所有与 S 相交的空间对象。

搜索算法（递归或迭代实现）

- ① 若 T 为非叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与查询矩形 S 相交，则递归搜索 E 指向的子节点。
- ② 若 T 为叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与 S 相交，则将 E 指向的实际对象加入候选集，最后可能需要进行精确的几何比较（过滤步骤）。

查询操作

R 树的查询通常为范围查询（窗口查询），给定一个查询矩形 S ，找出所有与 S 相交的空间对象。

搜索算法（递归或迭代实现）

- ① 若 T 为非叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与查询矩形 S 相交，则递归搜索 E 指向的子节点。
- ② 若 T 为叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与 S 相交，则将 E 指向的实际对象加入候选集，最后可能需要进行精确的几何比较（过滤步骤）。
- ③ 返回所有满足条件的对象。

查询操作

R 树的查询通常为范围查询（窗口查询），给定一个查询矩形 S ，找出所有与 S 相交的空间对象。

搜索算法（递归或迭代实现）

- ① 若 T 为非叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与查询矩形 S 相交，则递归搜索 E 指向的子节点。
- ② 若 T 为叶节点，检查 T 中每个条目 E 的 MBR。若 $E.MBR$ 与 S 相交，则将 E 指向的实际对象加入候选集，最后可能需要进行精确的几何比较（过滤步骤）。
- ③ 返回所有满足条件的对象。

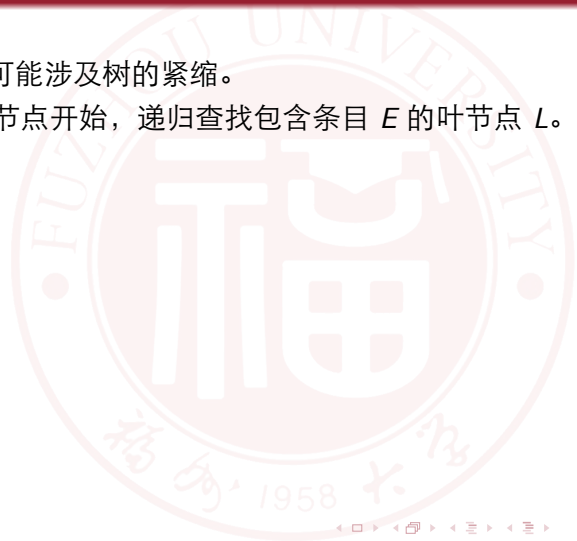
效率关键

查询性能高度依赖于节点 MBR 的重叠程度。重叠越小，搜索时需要遍历的无关分支就越少。R 树的变体（如 R^* 树）通过优化插入和分裂策略来减少重叠。

删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

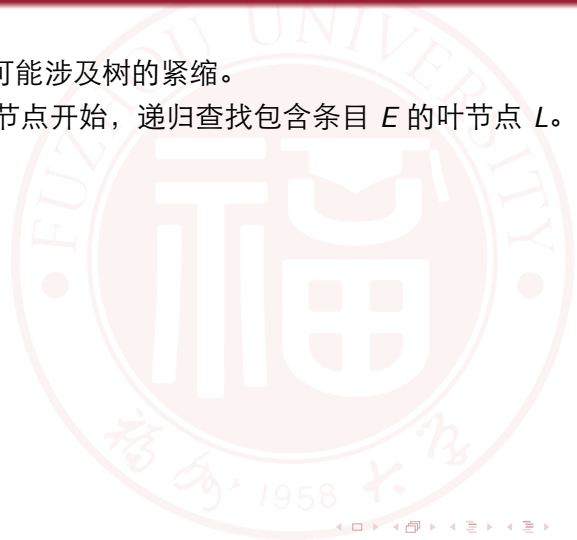
- ① 查找叶节点 (**FindLeaf**): 从根节点开始，递归查找包含条目 E 的叶节点 L 。



删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

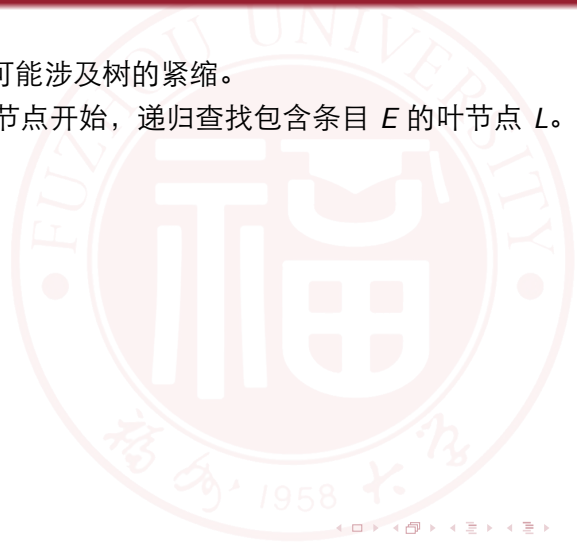
- ① 查找叶节点 (**FindLeaf**): 从根节点开始，递归查找包含条目 E 的叶节点 L 。
- ② 删除条目: 从 L 中删除 E 。



删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

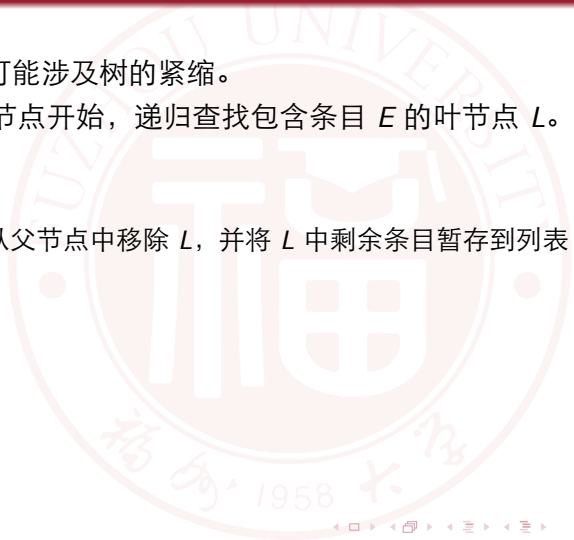
- ① 查找叶节点 (**FindLeaf**): 从根节点开始，递归查找包含条目 E 的叶节点 L 。
- ② 删除条目: 从 L 中删除 E 。
- ③ 紧缩树 (**CondenseTree**):



删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

- ① 查找叶节点 (**FindLeaf**): 从根节点开始，递归查找包含条目 E 的叶节点 L 。
- ② 删除条目: 从 L 中删除 E 。
- ③ 紧缩树 (**CondenseTree**):
 - 若 L 删除后条目数 $< m$ ，则从父节点中移除 L ，并将 L 中剩余条目暂存到列表 Q 中。



删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

- ① 查找叶节点 (**FindLeaf**): 从根节点开始，递归查找包含条目 E 的叶节点 L 。
- ② 删除条目: 从 L 中删除 E 。
- ③ 紧缩树 (**CondenseTree**):
 - 若 L 删除后条目数 $< m$ ，则从父节点中移除 L ，并将 L 中剩余条目暂存到列表 Q 中。
 - 向上回溯调整父节点的 MBR。此过程可能递归进行。

删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

- ① 查找叶节点 (**FindLeaf**): 从根节点开始，递归查找包含条目 E 的叶节点 L 。
- ② 删除条目: 从 L 中删除 E 。
- ③ 紧缩树 (**CondenseTree**):
 - 若 L 删除后条目数 $< m$ ，则从父节点中移除 L ，并将 L 中剩余条目暂存到列表 Q 中。
 - 向上回溯调整父节点的 MBR。此过程可能递归进行。
- ④ 重新插入孤立条目: 将紧缩过程中收集到列表 Q 中的所有条目，使用插入操作重新插入树中。这有助于优化树的结构。

删除操作

删除对象 E 的步骤比插入更复杂，可能涉及树的紧缩。

- ① 查找叶节点 (**FindLeaf**): 从根节点开始, 递归查找包含条目 E 的叶节点 L 。
- ② 删除条目: 从 L 中删除 E 。
- ③ 紧缩树 (**CondenseTree**):
 - 若 L 删除后条目数 $< m$, 则从父节点中移除 L , 并将 L 中剩余条目暂存到列表 Q 中。
 - 向上回溯调整父节点的 MBR。此过程可能递归进行。
- ④ 重新插入孤立条目: 将紧缩过程中收集到列表 Q 中的所有条目, 使用插入操作重新插入树中。这有助于优化树的结构。
- ⑤ 降低树高: 若根节点仅剩一个子节点, 则删除根节点, 将其子节点设为新根, 树高降低。

PostGIS 数据模型

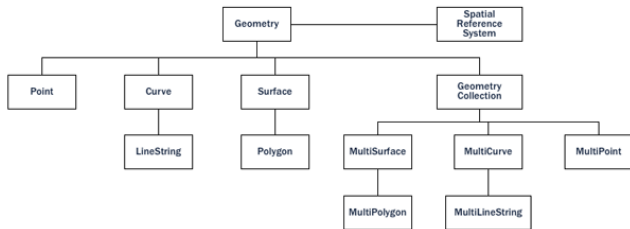
几何模型：

- 基于 OGC SFS 标准
- 内部格式：WKB 或压缩格式
- 轻量级表示减少元数据开销

空间参考系统：

- SRID 唯一标识坐标系
- 通过 spatial_ref_sys 表管理
- 支持动态投影转换

Geometry Hierarchy



几何类型层次结构

GeoMesa 发展背景

- 大数据挑战：物联网、社交媒体、轨迹数据等产生海量时空数据
- 传统方案不足：PostGIS 等单机系统在处理 PB 级数据时存在瓶颈
- 分布式需求：需要依托 HBase、Kafka 等分布式系统实现可扩展存储
- 核心目标：提供高效的时空索引、查询和分析能力，支持大规模应用

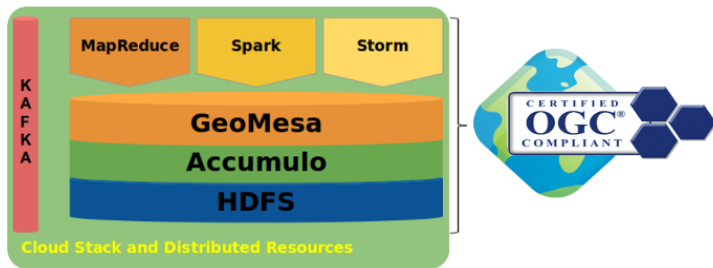
GeoMesa 定位

- 连接上层时空应用与底层大数据生态的“桥梁”
- 基于 LocationTech 开源项目，遵循 Apache 2.0 协议
- 广泛应用于京东、阿里、华为等企业

GeoMesa 系统架构

核心设计理念:

- 以时空索引为核心数据管理能力
- 兼容多种大数据组件（HBase、Accumulo、Spark、Kafka 等）
- 支持离线的数据管理和实时的数据导入



GeoMesa 整体架构图

GeoMesa 组件生态

支持的底层存储

- 列式存储：HBase、Accumulo、Google Bigtable
- 键值存储：Redis、Cassandra
- 流处理：Kafka

计算与分析框架

- 批处理：Apache Spark
- 流处理：Apache Flink
- 查询接口：GeoServer (WFS/WMS)、GeoTools

企业应用案例

京东 JUST 时空数据引擎、阿里 Ganos 空天数据库引擎、华为 CloudTable

如何存储轨迹

垂直存储

Key	Value	
Point Key	Spatio-temporal Properties	Other Properties
p_1	$lat_1 lng_1 t_1$	$oid_1 tid_1 \dots$
p_2	$lat_2 lng_2 t_2$	$oid_2 tid_2 \dots$
...	...	...
p_n	$lat_n lng_n t_n$	$oid_n tid_n \dots$

- 数据条目多；难压缩
- 一条记录轨迹信息不完整

水平存储

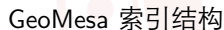
Key	Value			
Traj. Key	Spatio-temporal Properties	GPS Point List	Signature	Other Properties
$traj_1$	$mbr_1 t_{s1} t_{e1} p_{s1} p_{e1}$	$gzip(kryo(List<GPSPoint>))$	00010101	$oid_1 tid_1 \dots$
$traj_2$	$mbr_2 t_{s2} t_{e2} p_{s2} p_{e2}$	$gzip(kryo(List<GPSPoint>))$	10011001	$oid_2 tid_2 \dots$
...	...	...	...	...
$traj_m$	$mbr_m t_{sm} t_{em} p_{sm} p_{em}$	$gzip(kryo(List<GPSPoint>))$	10001001	$oid_m tid_m \dots$

- 数据条目少；便于压缩
- 一条记录包含完整轨迹信息

GeoMesa 索引结构

- Z2 索引的 RowKey 结构为 Shard + Z2(经度, 纬度) + FeatureID, 其中:

18



- Z2 索引的 RowKey 结构为 Shard + Z2(经度, 纬度) + FeatureID, 其中:
 - **Shard**: 用于数据分片, 均匀分布数据

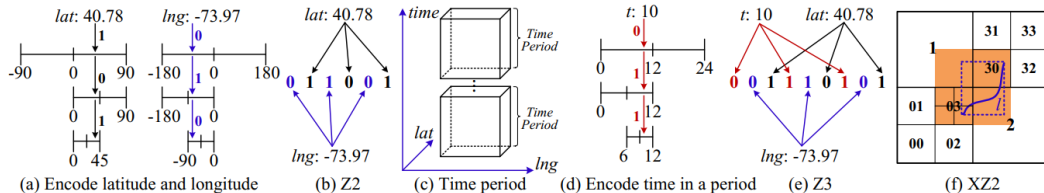
GeoMesa 索引结构

- Z2 索引的 RowKey 结构为 Shard + Z2(经度, 纬度) + FeatureID, 其中:
 - **Shard**: 用于数据分片, 均匀分布数据
 - **Z2(经度, 纬度)**: 通过 Z-order 曲线编码的空间位置
 - **FeatureID**: 唯一标识空间对象

GeoMesa 索引结构

- XZ2 索引的 RowKey 结构为 Shard + XZ2(MBX) + FeatureID, 其中:

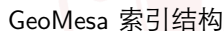
GeoMesa 原生索引



GeoMesa 索引结构

- Z3 的索引则是在 Z2 的基础上加上时间的二分编码

UNIVERSITY OF CALIFORNIA



- Z3 的索引则是在 Z2 的基础上加上时间的二分编码
- XZ3 的索引则是最小能覆盖形状的正方体的 Z3 编码的再编码

XZ2T 索引技术

发展历史

XZ2T 是 JUST³系统针对非点几何数据设计的核心时空索引策略，旨在高效管理如轨迹、多边形等线状或面状数据。其设计初衷是为了解决原生 GeoMesa 中 XZ3 索引在时空范围查询时空间过滤能力失效的关键问题。

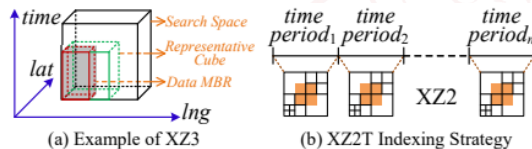
核心思想

将时间维度与空间维度解耦，采用“先分时，后索引”的策略。

- 时间分片：首先将连续的时间流划分为多个固定长度、互不重叠的时间区间。
- 空间索引：在每个时间区间内，为落入该区间的所有非点几何数据单独构建一个 XZ2 空间索引。

³R. Li et al., "JUST: JD Urban Spatio-Temporal Data Engine," 2020 IEEE 36th International Conference on Data Engineering (ICDE)

XZ2T 索引技术

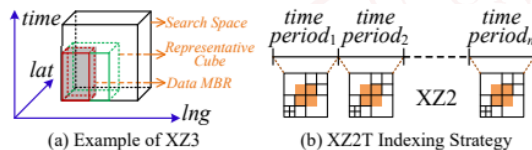


XZ3 与 XZ2T 的对比

索引键的构造

- Z2T: 时间片编号::Z2(经度, 纬度)

XZ2T 索引技术



XZ3 与 XZ2T 的对比

索引键的构造

- Z2T: 时间片编号::Z2(经度, 纬度)
- XZ2T: 时间片编号::XZ2(MBR)

XZT 用于 ID+ 时间范围查询。例如查询某个车辆在一段时间内的所有轨迹点。



利用网格划分记录轨迹形状，用于相似轨迹查相似轨迹的接触者。

XZ2⁺T 在 XZ2T 索引的基础上利用网格划分记录轨迹形状，用于相似轨迹查询。例如疫情期间排查相似轨迹的接触者。

其他索引技术

- XZT** 用于 ID+ 时间范围查询。例如查询某个车辆在一段时间内的所有轨迹点。
- XZ2+T** 在 XZ2T 索引的基础上利用网格划分记录轨迹形状，用于相似轨迹查询。例如疫情期间排查相似轨迹的接触者。
- TR** 将时间轴划分为短而固定长度的区间（如 30 分钟），称为时间周期。一个轨迹的时间范围 $[t_s, t_e]$ 可以用一个时间桶（Time Bin）来表示，该时间桶由从 t_s 所在的周期开始、连续跨越 N 个周期组成（ N 可配置）。该索引技术消除了死区，避免了冗余存储，并且通过相邻编码优化了查询的 I/O 效率。

其他索引技术

- XZT** 用于 ID+ 时间范围查询。例如查询某个车辆在一段时间内的所有轨迹点。
- XZ2+T** 在 XZ2T 索引的基础上利用网格划分记录轨迹形状，用于相似轨迹查询。例如疫情期间排查相似轨迹的接触者。
- TR** 将时间轴划分为短而固定长度的区间（如 30 分钟），称为时间周期。一个轨迹的时间范围 $[t_s, t_e]$ 可以用一个时间桶（Time Bin）来表示，该时间桶由从 t_s 所在的周期开始、连续跨越 N 个周期组成（ N 可配置）。该索引技术消除了死区，避免了冗余存储，并且通过相邻编码优化了查询的 I/O 效率。
- Tshape** 不再用单个矩形代表整个轨迹，而是允许使用一个由多个网格单元（Cell）任意组合形成的、不规则的“形状”来更精确地近似轨迹的实际空间分布。这个形状由位图（bitmap）表示，轨迹经过的单元置 1，否则置 0。Tshape 索引实现了对轨迹形状的细粒度、非矩形的精确描述，极大地减少了空间查询需要扫描的无关数据量。

③ 时空数据的处理

时空数据网格化

③ 时空数据的处理

时空数据网格化

- **基于几何的方法:** 使用欧氏距离或 Frechet 距离匹配点到最近道路



或 Frechet 距离匹配点到最近道路
声敏感

- 基于几何的方法: 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感

或 Frechet 距离匹配点到最近道路
声敏感
的连通性约束

- **基于几何的方法**: 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感
- **基于拓扑的方法**: 考虑道路网络的连通性约束

或 Frechet 距离匹配点到最近道路
声敏感
的连通性约束
高

- **基于几何的方法**: 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感
- **基于拓扑的方法**: 考虑道路网络的连通性约束
 - 结果更合理, 但计算复杂度较高

Journal of Management Studies, 46(7), 980–997.

- 或 Frechet 距离匹配点到最近道路
声敏感
的连通性约束
高
(HMM) 及其变体

地图匹配主要方法

- **基于几何的方法**: 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感
- **基于拓扑的方法**: 考虑道路网络的连通性约束
 - 结果更合理, 但计算复杂度较高
- **概率统计方法**: 隐马尔可夫模型 (HMM) 及其变体
 - 综合考虑 GPS 误差、道路拓扑和移动模式

地图匹配主要方法

- **基于几何的方法:** 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感
- **基于拓扑的方法:** 考虑道路网络的连通性约束
 - 结果更合理, 但计算复杂度较高
- **概率统计方法:** 隐马尔可夫模型 (HMM) 及其变体
 - 综合考虑 GPS 误差、道路拓扑和移动模式
 - **当前最主流和有效的方法之一**

地图匹配主要方法

- **基于几何的方法**: 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感
- **基于拓扑的方法**: 考虑道路网络的连通性约束
 - 结果更合理, 但计算复杂度较高
- **概率统计方法**: 隐马尔可夫模型 (HMM) 及其变体
 - 综合考虑 GPS 误差、道路拓扑和移动模式
 - 当前最主流和有效的方法之一
- **深度学习**方法: 使用神经网络学习匹配模式

rechet 距离匹配点到最近道路
感
通性约束
IM) 及其变体
移动模式
配模式
示能力

- 基于几何的方法: 使用欧氏距离或 Frechet 距离匹配点到最近道路
 - 简单高效, 但精度有限, 对噪声敏感
- 基于拓扑的方法: 考虑道路网络的连通性约束
 - 结果更合理, 但计算复杂度较高
- 概率统计方法: 隐马尔可夫模型 (HMM) 及其变体
 - 综合考虑 GPS 误差、道路拓扑和移动模式
 - 当前最主流和有效的方法之一
- 深度学习方法: 使用神经网络学习匹配模式
 - 如 L2MM 模型, 增强低频轨迹表示能力

统计模型，用于描述含有未知参数（即“隐藏”）
含一个不可观察的状态序列（马尔可夫链）和
程。

A, B, π):

- 有限集合
- 有限集合

$$a_{ij} = P(q_{t+1} = S_j | q_t = S_i)$$

矩阵, $b_j(k) = P(O_t = v_k | q_t = S_j)$

$$\pi_i = P(q_1 = S_i)$$

- 隐马尔可夫模型 (HMM) 是一种统计模型，用于描述含有未知参数（即“隐藏”状态）的马尔可夫过程。
- HMM 是一个双重随机过程，包含一个不可观察的状态序列（马尔可夫链）和一个与状态相关的可观察随机过程。
- 模型由五元组描述： $\lambda = (N, M, A, B, \pi)$:
 - $N = \{q_1, q_2, \dots, q_N\}$: 状态的有限集合
 - $M = \{v_1, v_2, \dots, v_M\}$: 观察值的有限集合
 - $A = \{a_{ij}\}$: 状态转移概率矩阵, $a_{ij} = P(q_{t+1} = S_j | q_t = S_i)$
 - $B = \{b_j(k)\}$: 观察值概率分布矩阵, $b_j(k) = P(O_t = v_k | q_t = S_j)$
 - $\pi = \{\pi_i\}$: 初始状态概率分布, $\pi_i = P(q_1 = S_i)$

- 给定观察序列 O ，调整模型参数 λ ，使得

实例：赌场欺诈问题

二、问题:

- 评估：该点数记录出现的概率有多大？
- 解码：哪些点是由灌铅骰子 B 掷出的？
- 学习：如何从数据中学习骰子特性及转换规律？

HMM 三个基本问题及其求解算法

问题类型	描述	求解算法
评估问题	计算观察序列概率 $P(O \lambda)$	前向算法 (Forward Algorithm) 后向算法 (Backward Algorithm)
解码问题	寻找最优状态序列 $\arg \max_Q P(Q O, \lambda)$	维特比算法 (Viterbi Algorithm)
学习问题	估计模型参数 $\arg \max_{\lambda} P(O \lambda)$	鲍姆-韦尔奇算法 (Baum-Welch Algorithm)

HMM 在地图匹配中的建模

- **状态空间:** 路网中的候选路段序列
- **观测序列:** GPS 轨迹点序列
- **解码问题:** 寻找最可能产生观测序列的隐藏状态序列

地图匹配的 HMM 建模

隐藏状态：车辆的实际道路位置

观测值：GPS 设备采集的坐标

目标: $\arg \max_{s_1, \dots, s_n} P(s_1, \dots, s_n | o_1, \dots, o_n)$

关键技术问题

- ① 如何计算观测概率（发射概率）
- ② 如何计算状态转移概率
- ③ 如何高效求解最优路径

核心公式：观测概率（发射概率）

- 定义: 给定真实位置下观测到当前 GPS 点的可能性
- 直观理解: GPS 点与候选路段距离越近, 概率越大
- $b_j(k)$ 或 $P(o_t|r_j)$ 表示 观测概率 (发射概率), 即当车辆实际位于路段 r_j 时, 观测到 GPS 点 o_k (或 o_t) 的概率

基于高斯分布的观测概率

$$b_j(k) = P(o_t = o_k | q_t = r_j) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{|o_t - r_j|^2}{2\sigma^2}}$$

- σ : GPS 误差的标准差，用于量化 GPS 测量噪声的大小，需要根据实际数据进行分析估计（通常取值在 5-50 米之间）
- $|o_t - r_j|$: GPS 观测点 o_t 到候选路段 r_j 上最近点（投影点）的地球表面最短距离（单位：米）

Figure 1

$$\frac{1}{\sum_{i=1}^n \text{Distance from } o_k \text{ to } r_i}$$

核心公式：状态转移概率

- **定义:** 车辆从一个路段转移到另一个路段的可能性
- **考虑因素:** 道路连通性、距离一致性、行驶速度
- a_{ij} 或 $P(r_j|r_i)$ 表示转移概率, 即车辆在时刻 t 位于路段 r_i 的条件下, 在下一时刻 $t+1$ 转移到路段 r_j 的概率。

综合考虑速度和距离的转移概率

$$a_{ij} = V(r_i \rightarrow r_j) \cdot F_t(r_i \rightarrow r_j)$$

$$F_t(r_i \rightarrow r_j) = \frac{\|o_t - o_{t+1}\|}{\|r_i - r_j\|}$$

维特比算法递推公式

$$\delta_t(i) = \max_j [\delta_{t-1}(j) \cdot a_{ji}] \cdot b_i(o_t) \quad (\text{累积最大概率})$$

$$\psi_t(i) = \arg \max_j [\delta_{t-1}(j) \cdot a_{ji}] \quad (\text{记录最优前驱节点})$$

符号/步骤	解释
$\delta_t(i)$	在时刻 t , 所有可能路径中到达状态 (路段) i 的最大概率。
$\psi_t(i)$	在时刻 t , 记录到达状态 i 的最优路径中, 前一个时刻 ($t-1$) 的状态是哪个。用于回溯构造最优路径。
初始化 ($t = 1$)	$\delta_1(i) = \pi_i \cdot b_i(o_1)$, $\psi_1(i) = 0$ 。其中 π_i 是初始状态概率, 常设为均匀分布或由第一个观测点决定。
递推 ($t = 2$ to T)	对每个时刻 t 和每个候选状态 i , 利用公式计算 $\delta_t(i)$ 和 $\psi_t(i)$ 。
终止	$P^* = \max_i \delta_T(i)$, 找到最终时刻概率最大的状态 i_T^* 。
路径回溯 ($t = T-1$ to 1)	$i_t^* = \psi_{t+1}(i_{t+1}^*)$, 从终点状态开始, 根据 ψ 回溯, 得到完整的最优路径序列 $(i_1^*, i_2^*, \dots, i_T^*)$ 。

HMM 算法总结

大致步骤

- ① 数据预处理：计算 GPS 轨迹点之间的距离
- ② 候选集选取：为每个 GPS 点选择可能的候选路段
- ③ 概率计算：发射概率计算、转移概率计算
- ④ 维特比算法：动态规划求解最优路径
- ⑤ 结果输出：输出匹配后的道路序列

后续优化方向

- 方向约束改进: 在计算发射概率时综合考虑距离和方向因素, 夹角偏差超过阈值的可以过滤
- 滑动窗口优化: 对于实时应用 (导航), 可以使用滑动窗口减少计算量

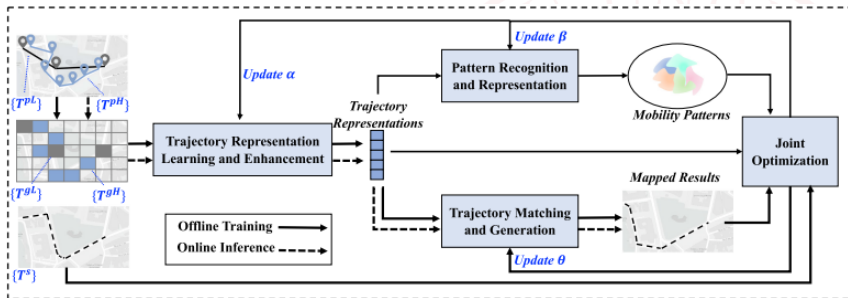
L2MM 算法：基于深度学习的低质量 GPS 轨迹地图匹配算法

- **噪声问题:** 城市峡谷效应、GPS 设备误差导致定位偏差
- **低频采样:** 能量和带宽限制导致采样间隔长达数分钟
- **非均匀采样:** 设备差异和通信故障导致采样时间间隔不均

传统方法局限性

- HMM 及其变体对低频采样敏感，性能下降明显
- 基于规则的方法难以适应复杂道路网络
- 早期深度学习方法对训练数据量敏感

L2MM 整体架构概览



L2MM 整体架构，包含四个核心组件

- 轨迹表示学习与增强: 学习高质量轨迹表示
- 模式识别与表示: 挖掘历史轨迹中的移动模式
- 轨迹匹配与生成: 将表示映射到道路序列
- 联合优化: 协同训练各组件

轨迹表示增强方法

解决低频轨迹信息稀疏问题

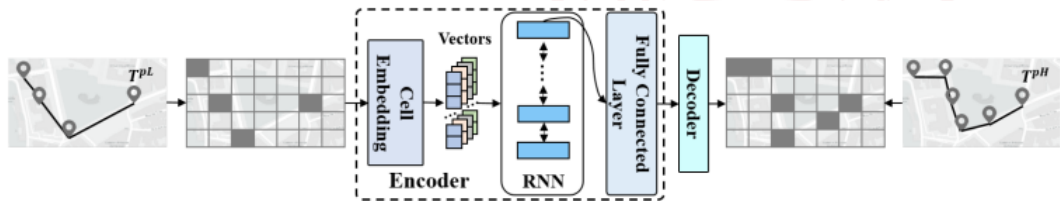
基于高频轨迹的增强

- 使用 Seq2Seq 模型学习低高频轨迹映射
- 编码器-解码器结构增强表示能力
- 双向 GRU 捕获前后向序列信息

基于数据分布的增强

- 变分自编码器 (VAE) 正则化潜在空间
- KL 散度约束表示分布接近高斯分布
- 提升模型泛化能力

表示增强技术细节



基于高频轨迹的表示增强示意图

- 编码器: 双向 GRU 处理网格轨迹, 输出上下文向量
- 解码器: 生成对应高频轨迹, 增强语

义信息

- **重参数化技巧**: 解决随机采样梯度回传问题

挖掘历史轨迹中的启发式线索

贝叶斯推理

- 潜在空间中识别 K 个典型移动模式
- 每个模式对应一个高斯分布成分
- 自动学习模式参数

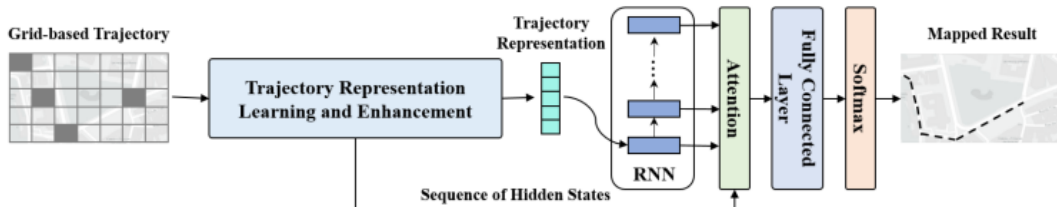
$$p_{\beta}(z|c) = \mathcal{N}(\mu_c, \sigma_c^2 I)$$

$$p_{\beta}(z) = \sum_{i=1}^K p_{\beta}(z|c) p_{\beta}(c)$$

- 计算轨迹属于各模式的后验概率
- 模式信息作为先验知识指导匹配

$$p_{\beta}(c|z) = \frac{p_{\beta}(c)p_{\beta}(z|c)}{p_{\beta}(z)}$$

轨迹匹配解码器设计



基于注意力机制的轨迹匹配生成过程

- **注意力机制**: 自适应关注相关隐藏状态
- **GRU 解码器**: 逐步生成道路段序列
- **Softmax 输出**: 预测每个时间步的道路段

$$p_{\theta}(e_i|e_{<i}, z) = \text{softmax}(F([s_i \oplus c_i]))$$

多组件协同训练

证据下界 (ELBO) 优化

$$\begin{aligned}\mathcal{L}_{ELBO} = & E_{q_\alpha(z|T^g)}[\log p_\theta(T^s|z)] \\ & - \lambda E_{q_\alpha(c|T^g)}[KL(q_\alpha(z|T^g)||p_\beta(z|c))] \\ & - \zeta E_{q_\alpha(c|T^g)}[KL(q_\alpha(c|z)||p_\beta(c))]\end{aligned}$$

- 第一项: 地图匹配准确率 (重构损失)
- 第二项: 表示分布与模式分布对齐
- 第三项: 模式后验与先验分布一致

③ 时空数据的处理

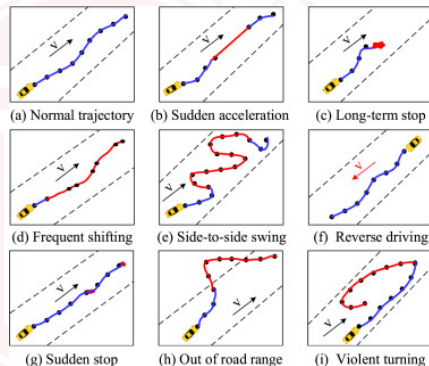
时空数据网格化

轨迹数据降噪概述

- **定义:** 从移动对象时空序列中剔除不可靠观测值, 保留真实运动模式
- **重要性:** 噪声直接影响后续分析准确性
 - 地图匹配误差
 - 行为识别偏差
 - 路况预测失准
- **核心挑战:** 平衡去噪效果与轨迹真实性

关键问题

如何有效区分真实运动模式与随机噪声？避免过度平滑导致特征丢失



非正常运动模式示意图

硬件误差

- GPS 定位漂移
- 传感器精度限制
- 时钟同步误差



[illegible]

© 2011 Blackwell Publishing Ltd *Journal of Internal Medicine* 270: 103–110

经典降噪方法：阈值法与滤波法

阈值法

基于物理约束识别异常:

- 速度閾値: $v_{max} < 200 \text{ km/h}$
- 加速度閾値: $|a| < 5 \text{ m/s}^2$
- 转角 (转向) 速率閾値

95 / 105

小波变换

多尺度分析分离噪声:

$$W_f(a, b) = \frac{1}{\sqrt{a}} \int f(t) \psi\left(\frac{t-b}{a}\right) dt$$

适用非平稳序列

现代降噪方法：小波变换与奇异谱分析

小波变换

多尺度分析分离噪声:

$$W_f(a, b) = \frac{1}{\sqrt{a}} \int f(t) \psi\left(\frac{t-b}{a}\right) dt$$

适用非平稳序列

奇异谱分析 (SSA)

- ① 轨迹矩阵构建
 - ② SVD 分解
 - ③ 分组重构
 - ④ 对角平均

- **GAN**: 学习干净轨迹分布
- 扩散模型: 去噪生成高质量轨迹
- **Transformer**: 捕捉长期依赖

现代降噪方法：小波变换与奇异谱分析

小波变换

多尺度分析分离噪声:

$$W_f(a, b) = \frac{1}{\sqrt{a}} \int f(t) \psi\left(\frac{t-b}{a}\right) dt$$

适用非平稳序列

奇异谱分析 (SSA)

- ① 轨迹矩阵构建
 - ② SVD 分解
 - ③ 分组重构
 - ④ 对角平均

机器学习方法

- **GAN**: 学习干净轨迹分布
- 扩散模型: 去噪生成高质量轨迹
- **Transformer**: 捕捉长期依赖

sEMA 滤波

对称指数移动平均:

$$y_t = \alpha x_t + (1 - \alpha)(y_{t-1} + y_{t+1})/2$$

NGSIM 数据集验证有效

前沿趋势与研究挑战

前沿趋势

- 端到端学习: 联合优化去噪与下游任务
- 轻量化部署: 边缘计算实时处理
- 隐私保护: 差分隐私联邦学习
- 多模态融合: GPS+IMU+ 视觉数据

前沿趋势

- 端到端学习: 联合优化去噪与下游任务
- 轻量化部署: 边缘计算实时处理
- 隐私保护: 差分隐私联邦学习
- 多模态融合: GPS+IMU+ 视觉数据

API 服务

- 高德猎鹰轨迹服务
- 百度鹰眼轨迹管理
- 云端纠偏与降噪

前沿趋势与研究挑战

前沿趋势

- 端到端学习: 联合优化去噪与下游任务
- 轻量化部署: 边缘计算实时处理
- 隐私保护: 差分隐私联邦学习
- 多模态融合: GPS+IMU+ 视觉数据

API 服务

- 高德猎鹰轨迹服务
- 百度鹰眼轨迹管理
- 云端纠偏与降噪

研究挑战

- 评估标准: 缺乏统一基准数据集
- 可解释性: 深度学习黑箱问题
- 泛化能力: 跨场景适应性
- 实时性: 大规模流数据处理

③ 时空数据的处理

时空数据网格化

驻留点检测概述

定义

驻留点 (Stay Point) 是移动对象在较小地理范围内持续停留一段时间所形成的轨迹片段, 由一系列空间上接近、时间跨度较长的连续 GPS 点组成。

- **空间特征:** 点位在有限地理区域内密集分布
- **时间特征:** 总停留时间超过常规移动通过所需时间
- **语义价值:** 蕴含丰富的用户行为信息 (购物、就餐、工作等)

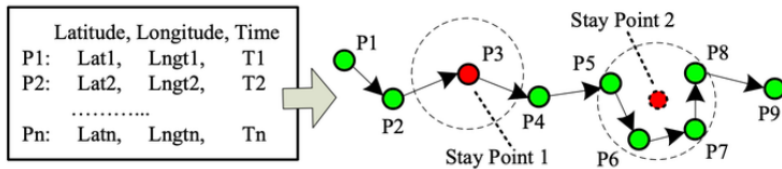


Figure 1. GPS logs and stay points

CSDN @UOI-LIUWJ

◀ ▶ 🔍 ↺ ↻ 🔍

空间阈值检测

从锚点 (anchor) 开始, 寻找连续 n 个与锚点距离小于 D_{max} 的 GPS 点:

$$dist(anchor, p_i) < D_{max}, \forall i \in [1, n]$$

空间阈值检测

从锚点 (anchor) 开始, 寻找连续 n 个与锚点距离小于 D_{max} 的 GPS 点:

$$dist(anchor, p_i) < D_{max}, \forall i \in [1, n]$$

时间阈值检测

检查候选点集的时间跨度是否超过 T_{min} :

$$t_n - t_{anchor} > T_{min}$$

空间阈值检测

$$dist(anchor, p_i) < D_{max}, \forall i \in [1, n]$$

时间阈值检测

$$t_n - t_{anchor} > T_{min}$$

算法流程

- 1 设置锚点为轨迹起点
- 2 空间范围检测
- 3 时间跨度检测
- 4 锚点推进
- 5 重复直至轨迹结束

空间阈值检测

从锚点 (anchor) 开始, 寻找连续 n 个与锚点距离小于 D_{max} 的 GPS 点:

$$dist(anchor, p_i) < D_{max}, \forall i \in [1, n]$$

时间阈值检测

检查候选点集的时间跨度是否超过 T_{min} :

$$t_n - t_{anchor} > T_{min}$$

算法流程

- 1 设置锚点为轨迹起点
- 2 空间范围检测
- 3 时间跨度检测
- 4 锚点推进
- 5 重复直至轨迹结束

参数设置

- D_{max} : 通常 50-200 米
- T_{min} : 通常 10-30 分钟
- 需根据具体场景调整

锚点推进策略对比

基础算法

- 检测到驻留点后，锚点跳至驻留点下一个点
- 可能遗漏重叠驻留点
- 计算效率较高

锚点推进策略对比

基础算法

- 检测到驻留点后，锚点跳至驻留点下一个点
- 可能遗漏重叠驻留点
- 计算效率较高

适用场景

- 高采样率轨迹数据
- 实时检测需求
- 计算资源有限

密度合并算法

- 逐点推进锚点，检测所有可能驻留点
- 合并相互重叠的驻留点
- 检测更全面，计算量较大

锚点推进策略对比

基础算法

- 检测到驻留点后，锚点跳至驻留点下一个点
- 可能遗漏重叠驻留点
- 计算效率较高

适用场景

- 高采样率轨迹数据
- 实时检测需求
- 计算资源有限

密度合并算法

- 逐点推进锚点，检测所有可能驻留点
- 合并相互重叠的驻留点
- 检测更全面，计算量较大

适用场景

- 低采样率数据
- 对准确性要求高
- 离线分析任务

DBSCAN 扩展

- 将时间维度融入距离度量
- 时空聚类避免误检频繁经过区域
- 参数: eps(距离), min_samples(最小点数)

基于聚类的高级检测方法

DBSCAN 扩展

- 将时间维度融入距离度量
- 时空聚类避免误检频繁经过区域
- 参数: eps(距离), min_samples(最小点数)

ST-CFSFDP 算法

针对低采样信令数据改进:

- 扩展至三维时空空间 (经度、纬度、时间)
- 定义高密度时间间隔

对任豆样 夕家麻好具任亡性而好

基于聚类的高级检测方法

DBSCAN 扩展

- 将时间维度融入距离度量
- 时空聚类避免误检频繁经过区域
- 参数: eps(距离), min_samples(最小点数)

ST-CFSFDP 算法

针对低采样信令数据改进:

- 扩展至三维时空空间 (经度、纬度、时间)
- 定义高密度时间间隔

对低采样 各窗长度自适应性好

三维密度计算

$$\rho(x, y, t) = \sum_i K\left(\frac{d_{space}}{h_s}\right) K\left(\frac{d_{time}}{h_t}\right)$$

其中 h_s, h_t 分别为空间和时间带宽

DBSCAN 扩展

- 将时间维度融入距离度量
- 时空聚类避免误检频繁经过区域
- 参数: eps(距离), min_samples(最小点数)

ST-CFSFDP 算法

针对低采样信令数据改进:

- 扩展至三维时空空间 (经度、纬度、时间)
- 定义高密度时间间隔

对低采样 各密度均呈正态分布

三维密度计算

$$\rho(x, y, t) = \sum_i K\left(\frac{d_{space}}{h_s}\right) K\left(\frac{d_{time}}{h_t}\right)$$

其中 h_s, h_t 分别为空间和时间带宽

算法优势对比

聚类方法性能对比

方法	准确性	计算效率	参数敏感性
阈值法	中等	高	高
DBSCAN	较高	中等	中等
ST-CFSFDP	高	低	低

驻留点检测的挑战与应对策略

数据质量问题

- **低采样密度:** 手机信令数据采样间隔大
- **定位精度差:** 基站定位误差较大
- **GPS 漂移:** 静止车辆速度值非零



驻留点检测的挑战与应对策略

数据质量问题

- **低采样密度:** 手机信令数据采样间隔大
- **定位精度差:** 基站定位误差较大
- **GPS 漂移:** 静止车辆速度值非零

应对策略

- ST-CFSFDP 算法处理低采样数据
- 数据预处理消除震荡轨迹
- 数据驱动法确定速度阈值

驻留点检测的挑战与应对策略

数据质量问题

- **低采样密度:** 手机信令数据采样间隔大
- **定位精度差:** 基站定位误差较大
- **GPS 漂移:** 静止车辆速度值非零

应对策略

- ST-CFSFDP 算法处理低采样数据
- 数据预处理消除震荡轨迹
- 数据驱动法确定速度阈值

行为复杂性

- 临时驻留: 等红灯 (临时) vs 装卸货物 (真实)
- 参数主观性: 阈值设置依赖经验
- 多模式停留: 不同停留行为特征差异大

驻留点检测的挑战与应对策略

数据质量问题

- **低采样密度**: 手机信令数据采样间隔大
- **定位精度差**: 基站定位误差较大
- **GPS 漂移**: 静止车辆速度值非零

应对策略

- ST-CFSFDP 算法处理低采样数据
- 数据预处理消除震荡轨迹
- 数据驱动法确定速度阈值

行为复杂性

- 临时驻留: 等红灯 (临时) vs 装卸货物 (真实)
- 参数主观性: 阈值设置依赖经验
- 多模式停留: 不同停留行为特征差异大

应对策略

- 结合路网信息排除道路上停留
- 关联 POI 信息进行语义筛选
- 混合模型拟合确定多级时间阈值

事件分析

- 出租车轨迹: 分析加油等待时间
- 快递员轨迹: 推断运单妥投时间
- 危化品车辆: 发现异常停留, 辅助安全监管

事件分析

- 出租车轨迹: 分析加油等待时间
- 快递员轨迹: 推断运单妥投时间
- 危化品车辆: 发现异常停留, 辅助安全监管

加油站分析案例

- 驻留点与加油站 POI 关联
- 分析各时段平均等待时间
- 推荐最佳加油地点和时间

地点发现

- 聚类驻留点发现未记录的兴趣地点
- 出租车候客点推荐
- 危化品车辆异常地点检测

① 时空数据的采集

② 时空数据的存储

③ 时空数据的处理

轨迹地图匹配

轨迹数据降噪

驻留点检测

时空数据网格化